



From Data to Solutions · Weekend 6

Monte Carlo Tree Search

How an agent picks its next move

Carlos Cotrini

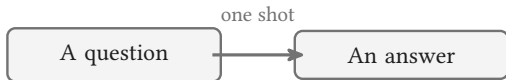


Section 1

Agents, and why they look like games

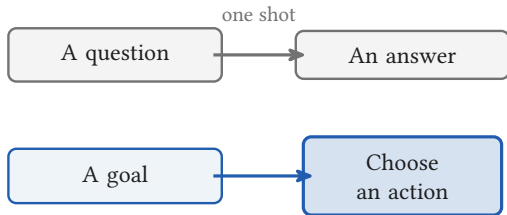
1

From answering to acting



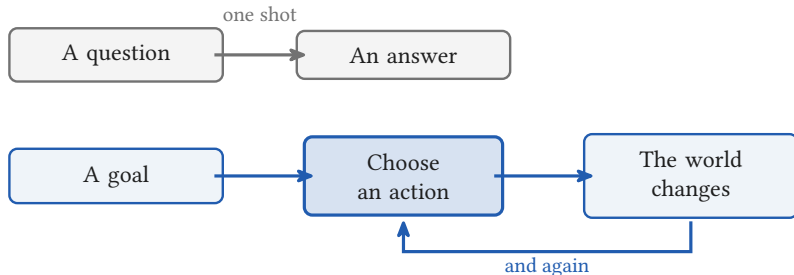
- A model that **answers** is judged on one output

From answering to acting



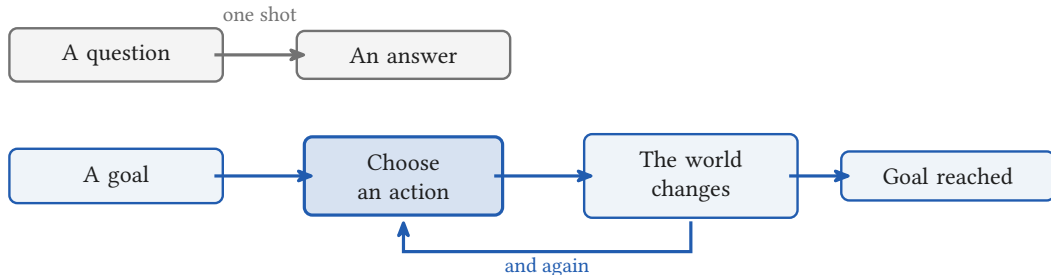
- A model that **answers** is judged on one output
- An **agent** is judged on whether the goal is reached

From answering to acting



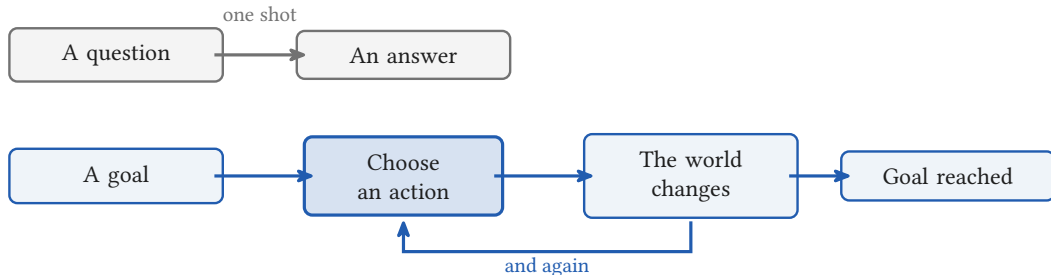
- A model that **answers** is judged on one output
- An **agent** is judged on whether the goal is reached

From answering to acting



- A model that **answers** is judged on one output
- An **agent** is judged on whether the goal is reached

From answering to acting



- A model that **answers** is judged on one output
- An **agent** is judged on whether the goal is reached
- So every step is a **decision**, and a wrong one costs the whole task

Where this is already running

Software

read a repo,
edit files, run
the tests, open
a pull request

Where this is already running

Software

read a repo,
edit files, run
the tests, open
a pull request

Web tasks

search a site, fill a
form, book or buy

Where this is already running

Software

read a repo,
edit files, run
the tests, open
a pull request

Web tasks

search a site, fill a
form, book or buy

Research

search, read,
cross-check, write
the summary

Where this is already running

Software

read a repo,
edit files, run
the tests, open
a pull request

Web tasks

search a site, fill a
form, book or buy

Research

search, read,
cross-check, write
the summary

Operations

triage a ticket,
look things up,
act or escalate

Where this is already running

Software

read a repo,
edit files, run
the tests, open
a pull request

Web tasks

search a site, fill a
form, book or buy

Research

search, read,
cross-check, write
the summary

Operations

triage a ticket,
look things up,
act or escalate

- All four are [sequences of actions](#), not single answers

Where this is already running

Software

read a repo,
edit files, run
the tests, open
a pull request

Web tasks

search a site, fill a
form, book or buy

Research

search, read,
cross-check, write
the summary

Operations

triage a ticket,
look things up,
act or escalate

- All four are **sequences of actions**, not single answers
- All four are judged at the **end**: did the tests pass, is the table booked, is the summary right

These tasks are games

the task	a position	a move	you have won when
Chess	the board right now	a legal move	checkmate

These tasks are games

the task	a position	a move	you have won when
Chess	the board right now	a legal move	checkmate
Fixing a bug	the repo, and a failing test	an edit, or a command	the tests pass

These tasks are games

the task	a position	a move	you have won when
Chess	the board right now	a legal move	checkmate
Fixing a bug	the repo, and a failing test	an edit, or a command	the tests pass
Booking a table	the page you are on	a click, or something typed	the booking is confirmed

These tasks are games

the task	a position	a move	you have won when
Chess	the board right now	a legal move	checkmate
Fixing a bug	the repo, and a failing test	an edit, or a command	the tests pass
Booking a table	the page you are on	a click, or something typed	the booking is confirmed
Buying the right item	the search results	search, or click a product	it matches what was asked

These tasks are games

the task	a position	a move	you have won when
Chess	the board right now	a legal move	checkmate
Fixing a bug	the repo, and a failing test	an edit, or a command	the tests pass
Booking a table	the page you are on	a click, or something typed	the booking is confirmed
Buying the right item	the search results	search, or click a product	it matches what was asked

In all four, one move is rarely right or wrong on its own. It is right if where it leads is good.



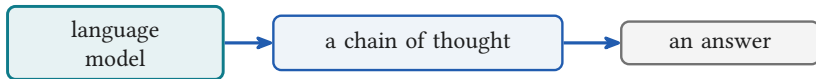
Section 2

How a language model becomes an agent

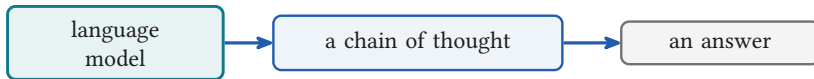
2

**How do we find the right
move
for each task?**

Where the good moves are going to come from



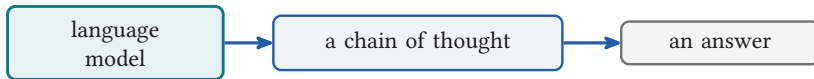
Where the good moves are going to come from



AIME 2024, pass@1

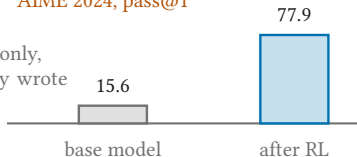


Where the good moves are going to come from

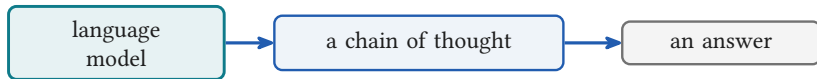


AIME 2024, pass@1

At 08:00: reward the final answer only,
and the reasoning appears. Nobody wrote
those chains.

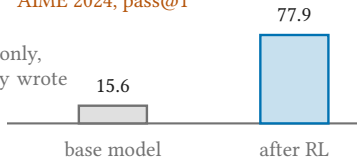


Where the good moves are going to come from



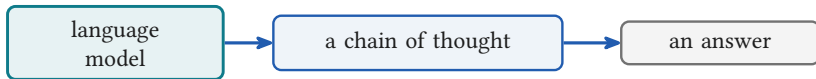
AIME 2024, pass@1

At 08:00: reward the final answer only, and the reasoning appears. Nobody wrote those chains.



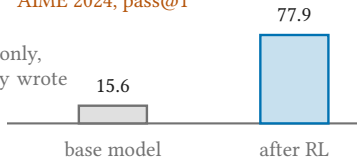
- A model that reasons before it answers is what we will search with

Where the good moves are going to come from



AIME 2024, pass@1

At 08:00: reward the final answer only, and the reasoning appears. Nobody wrote those chains.

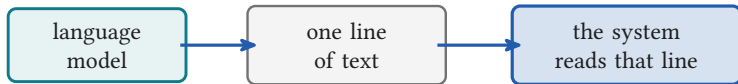


- A model that reasons before it answers is what we will search with
- But a chain of thought is **still just text**. Nothing has happened yet

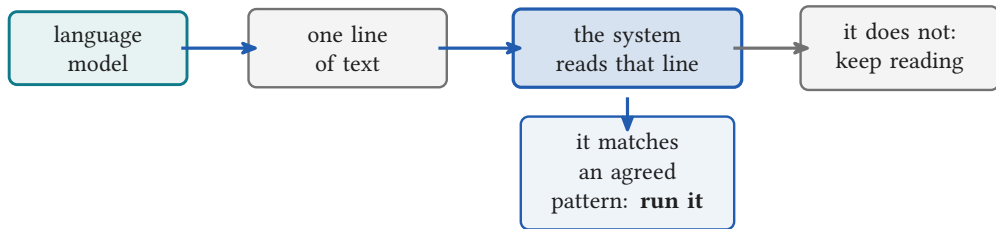
How a model gets to act



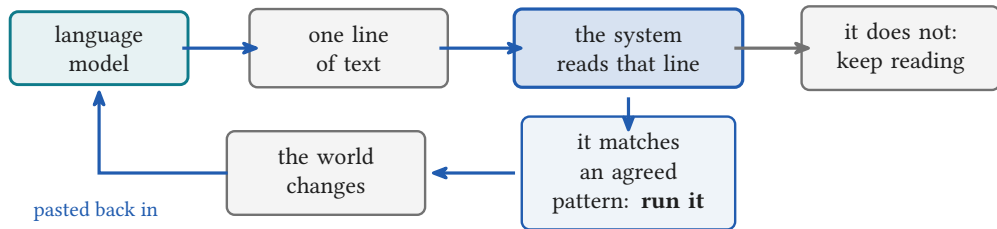
How a model gets to act



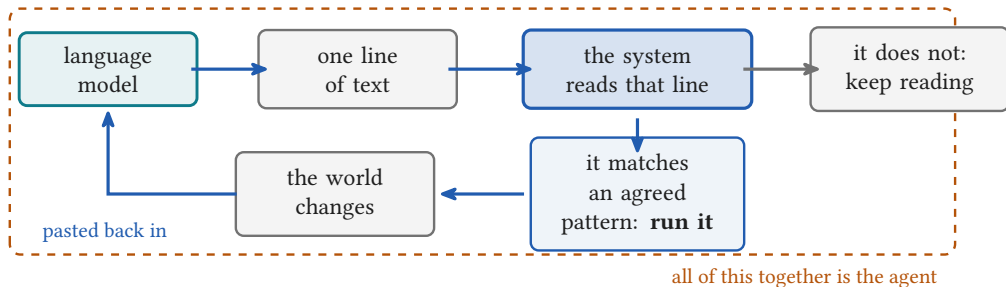
How a model gets to act



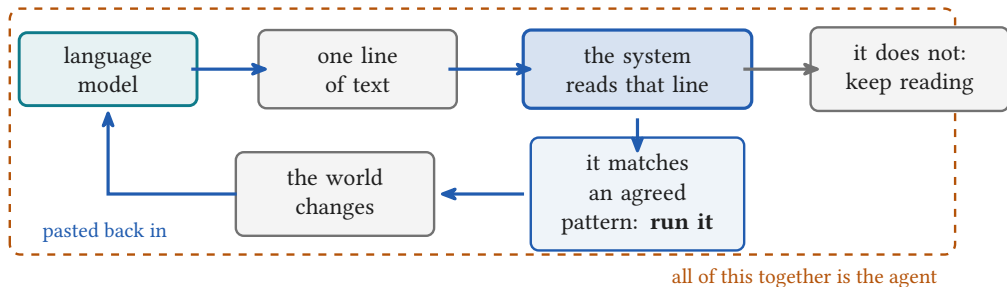
How a model gets to act



How a model gets to act

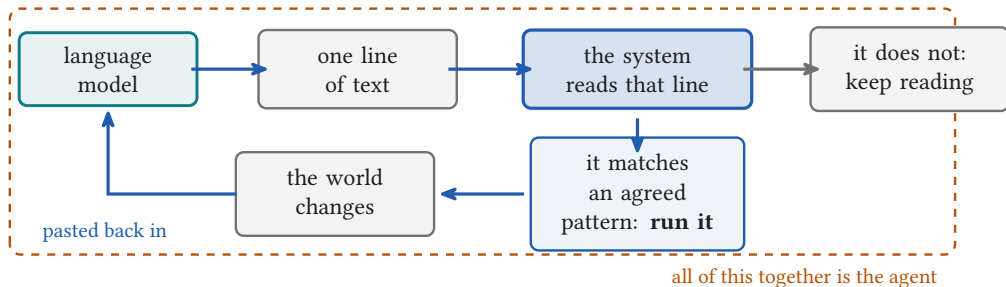


How a model gets to act



- The model still only ever emits **text**. It touches nothing

How a model gets to act



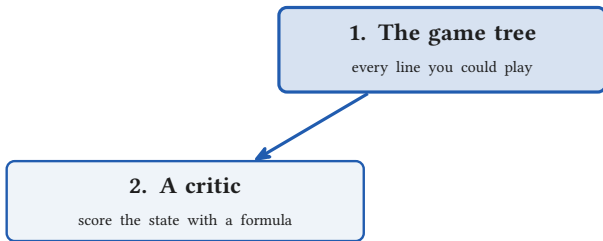
- The model still only ever emits **text**. It touches nothing
- The system agrees **in advance** which text counts as a command

Four things, and the last one is the other three

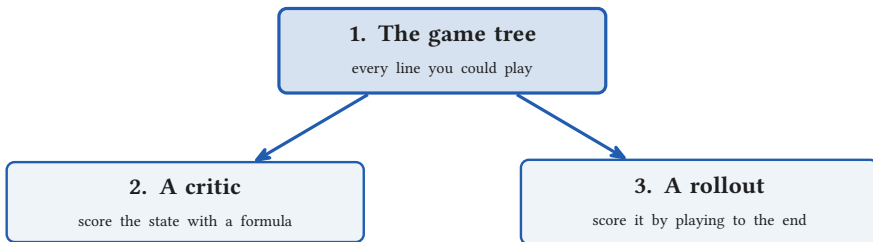
1. The game tree

every line you could play

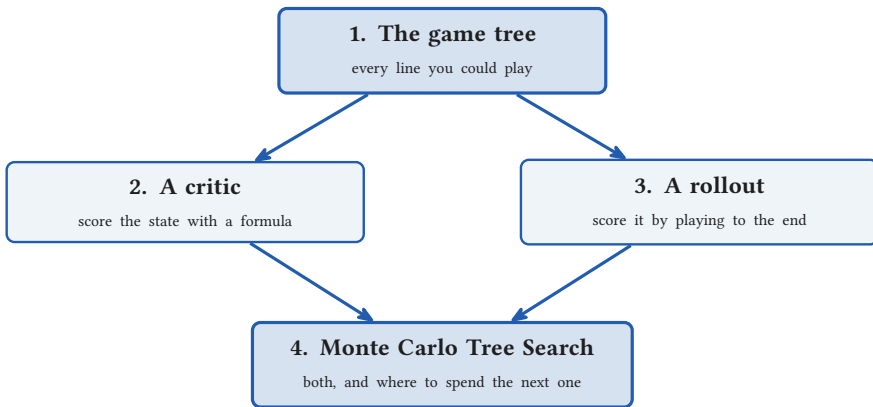
Four things, and the last one is the other three



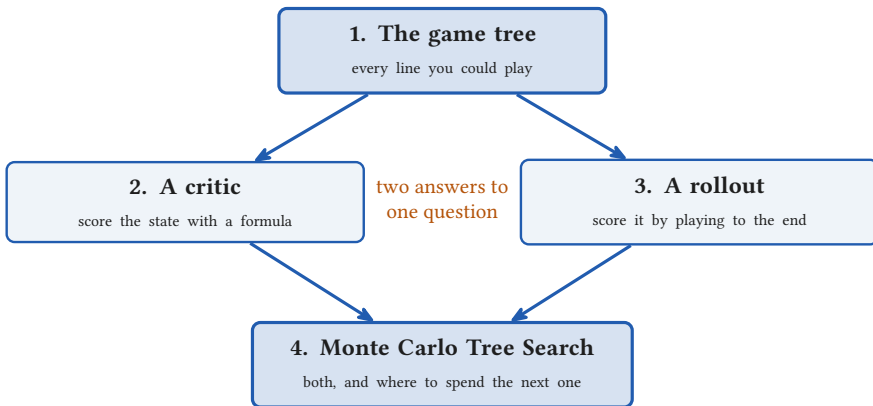
Four things, and the last one is the other three



Four things, and the last one is the other three



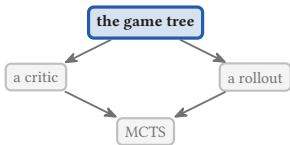
Four things, and the last one is the other three





Section 3

The game tree



3

One trajectory, start to finish



One trajectory, start to finish



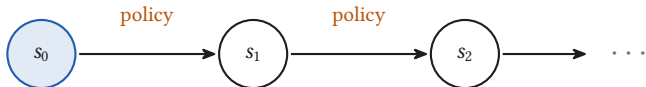
- A **policy** is anything that picks a move: a fixed rule, a random choice, or a language model

One trajectory, start to finish



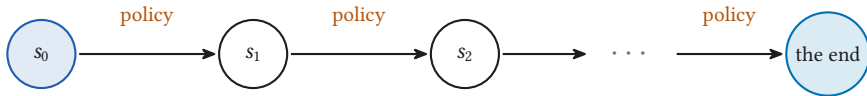
- A **policy** is anything that picks a move: a fixed rule, a random choice, or a language model

One trajectory, start to finish



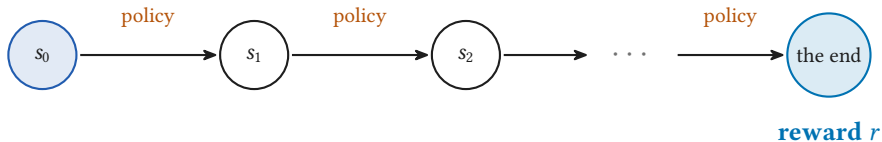
- A **policy** is anything that picks a move: a fixed rule, a random choice, or a language model

One trajectory, start to finish



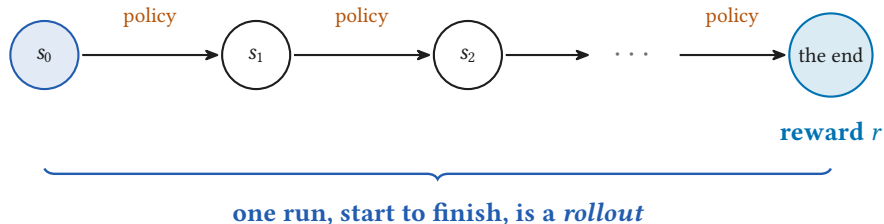
- A **policy** is anything that picks a move: a fixed rule, a random choice, or a language model

One trajectory, start to finish



- A **policy** is anything that picks a move: a fixed rule, a random choice, or a language model
- The reward arrives only at the **end**, when the game is over

One trajectory, start to finish



- A **policy** is anything that picks a move: a fixed rule, a random choice, or a language model
- The reward arrives only at the **end**, when the game is over
- No looking ahead, no comparing. It observes, it picks, it moves on.

One bad action ends the run

follow one trajectory



One bad action ends the run

follow one trajectory



one action goes wrong

One bad action ends the run

follow one trajectory



one action goes wrong

the run is lost,
and nothing after
it recovers

One bad action ends the run

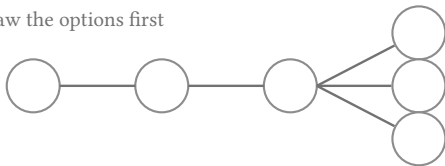
follow one trajectory



one action goes wrong

the run is lost,
and nothing after
it recovers

or draw the options first



One bad action ends the run

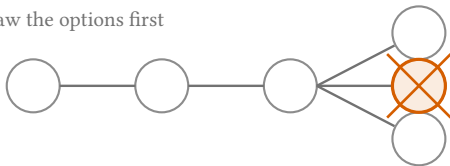
follow one trajectory



one action goes wrong

the run is lost,
and nothing after
it recovers

or draw the options first



found before it is played

One bad action ends the run

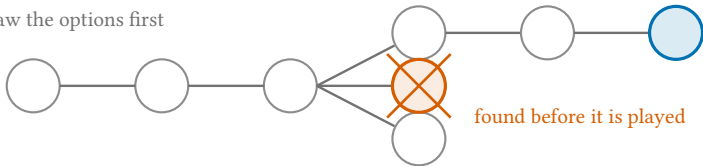
follow one trajectory



one action goes wrong

the run is lost,
and nothing after
it recovers

or draw the options first



found before it is played

the run survives

One bad action ends the run

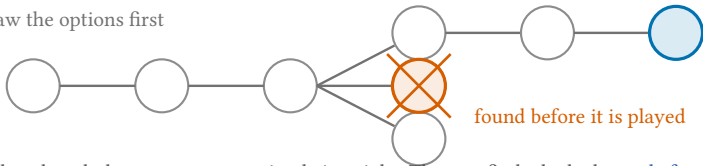
follow one trajectory



one action goes wrong

the run is lost,
and nothing after
it recovers

or draw the options first



found before it is played

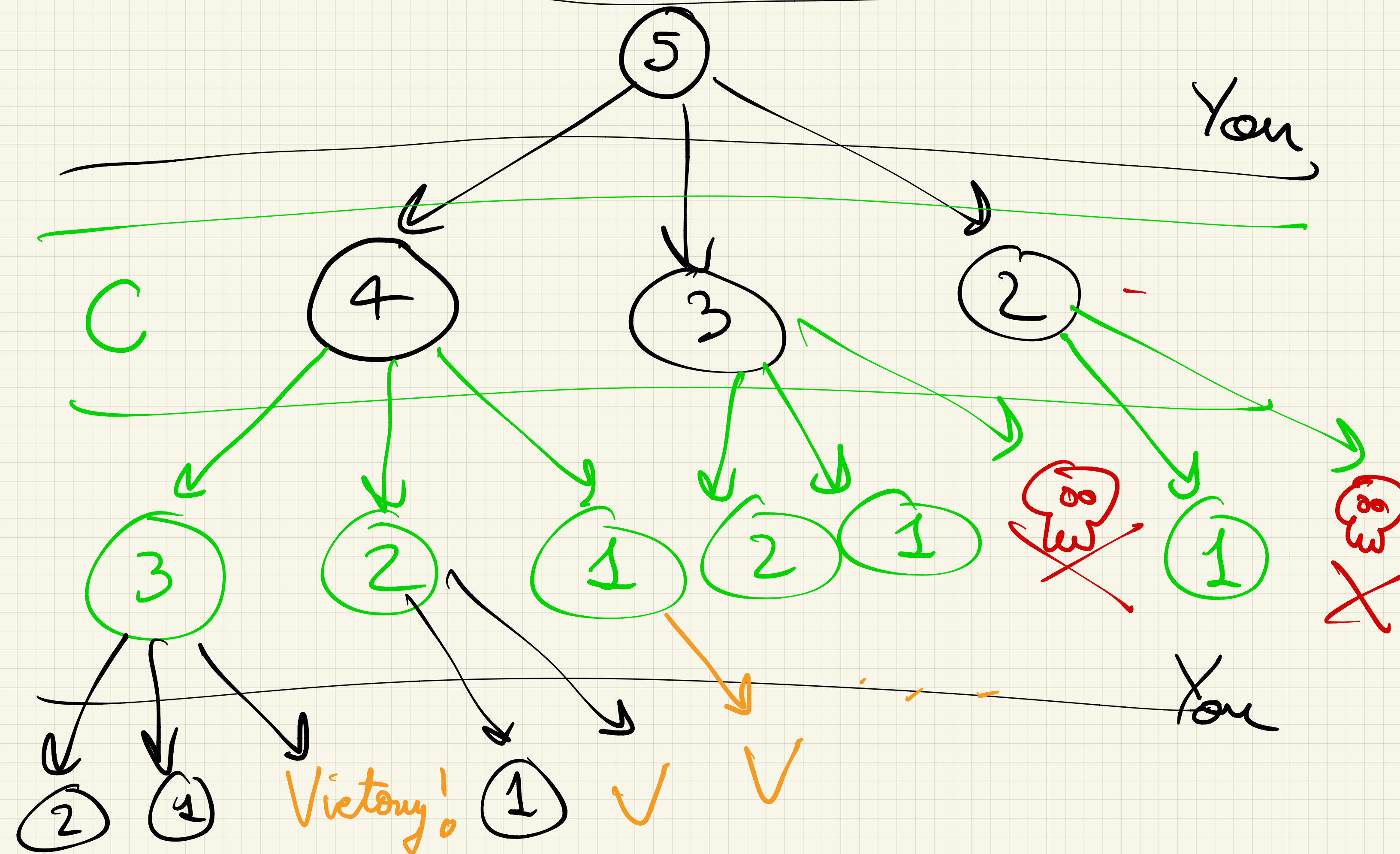
the run survives

One rollout stakes the whole run on every action being right. The tree finds the bad ones **before** they are played.

Five matches

Five matches on the table. On your turn, take one, two or three.
Whoever takes the last match wins.





Five matches

Five matches on the table. On your turn, take one, two or three.
Whoever takes the last match wins.



two players, and nothing is hidden

Five matches

Five matches on the table. On your turn, take one, two or three.
Whoever takes the last match wins.



two players, and nothing is hidden

so every way the game can go is already decided
by the rules, and we can write it down

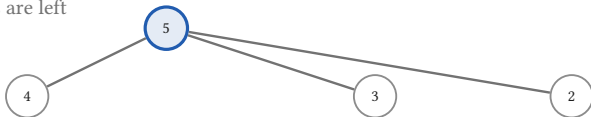
The whole game tree, for once

each circle is how many matches are left



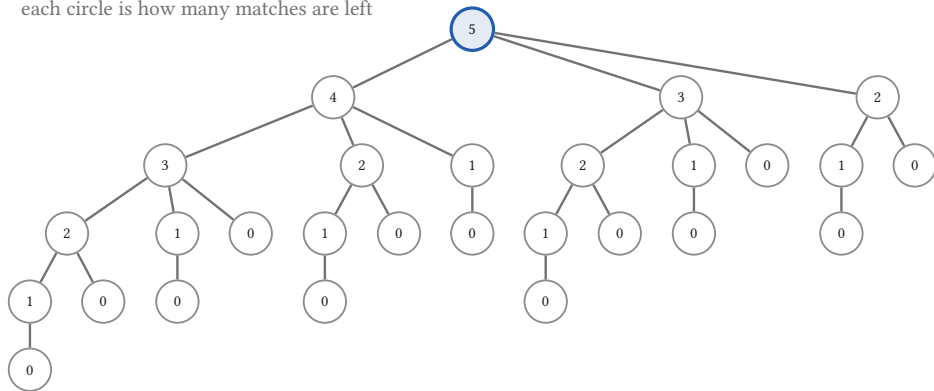
The whole game tree, for once

each circle is how many matches are left



The whole game tree, for once

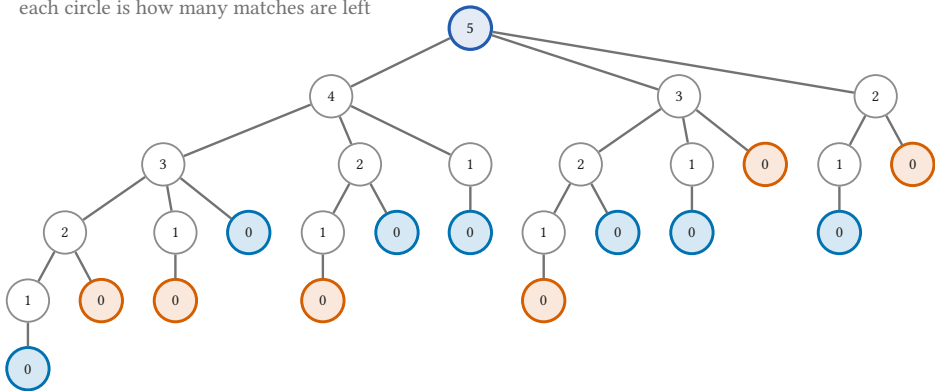
each circle is how many matches are left





The whole game tree, for once

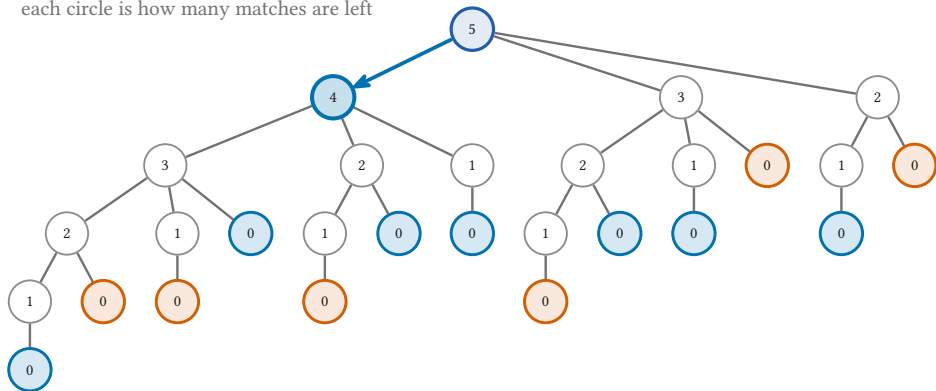
each circle is how many matches are left



blue: the player who moved first took the last match

The whole game tree, for once

each circle is how many matches are left



blue: the player who moved first took the last match

and exactly one first move wins: take one

five matches

28 nodes

five matches

28 nodes

seven matches

96 nodes

five matches

28 nodes

seven matches

96 nodes

chess

about 10^{120} games

1 00000

20

five matches	28 nodes
seven matches	96 nodes
chess	about 10^{120} games

so from here on, we never see the whole tree again

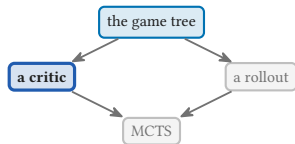
**You ran out of budget partway
down.
How good is where you are
standing?**

The next two sections are two different answers.



Section 4

Scoring a state with a critic

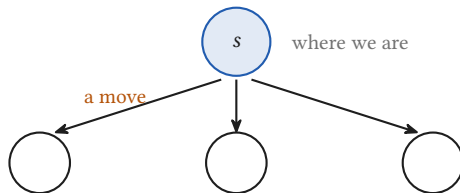


Look ahead, score what you find, take the best



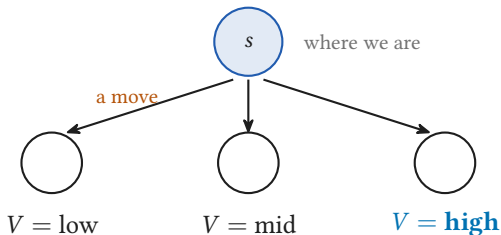
where we are

Look ahead, score what you find, take the best



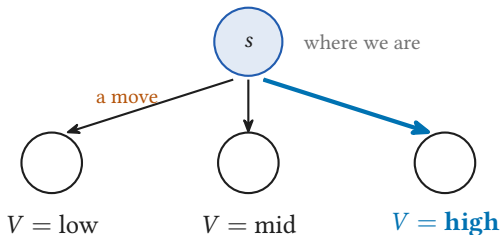
- List the moves, and the state each one leads to

Look ahead, score what you find, take the best



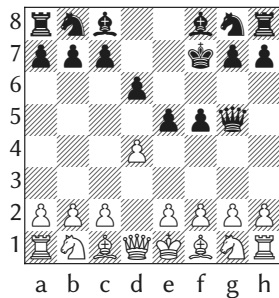
- List the moves, and the state each one leads to
- Give each of those states a **score**

Look ahead, score what you find, take the best



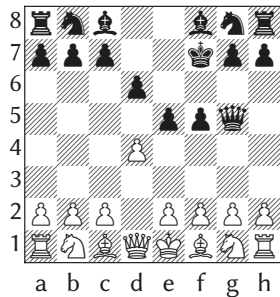
- List the moves, and the state each one leads to
- Give each of those states a **score**
- Play the move with the best score

One position, three moves



White to move

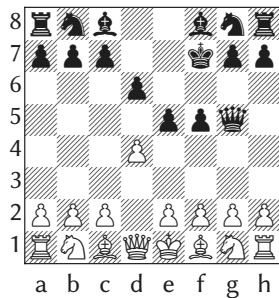
One position, three moves



White to move

The Black queen sits alone on g5

One position, three moves

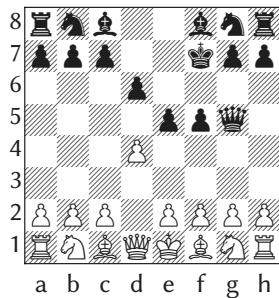


White to move

Take the queen

The Black queen sits alone on g5

One position, three moves



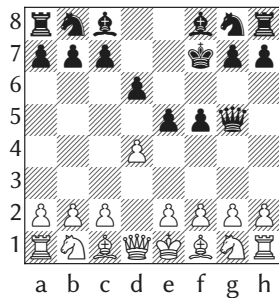
White to move

The Black queen sits alone on g5

Take the queen

A quiet
pawn move

One position, three moves



White to move

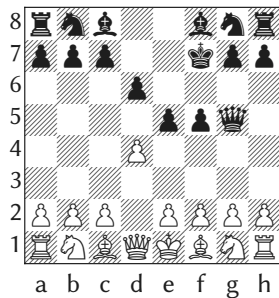
The Black queen sits alone on g5

Take the queen

A quiet
pawn move

Stop one
square short

One position, three moves



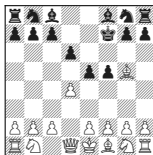
White to move

The Black queen sits alone on g5

Take the queen

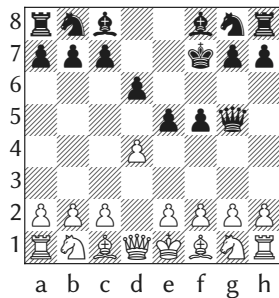
A quiet
pawn move

Stop one
square short



s1

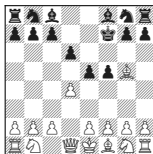
One position, three moves



White to move

The Black queen sits alone on g5

Take the queen



s_1

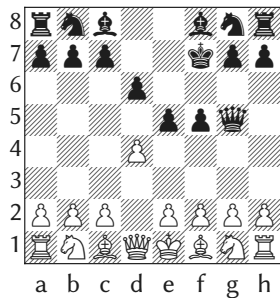
A quiet
pawn move



s_2

Stop one
square short

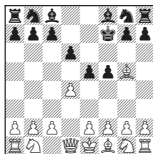
One position, three moves



White to move

The Black queen sits alone on g5

Take the queen



s_1

A quiet
pawn move



s_2

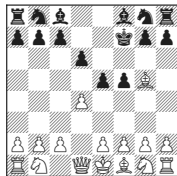
Stop one
square short



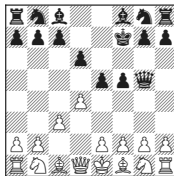
s_3

the pawn on e5
can eat the bishop

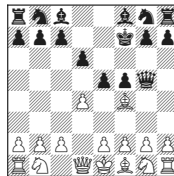
A critic scores each option



S1

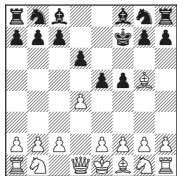


S2

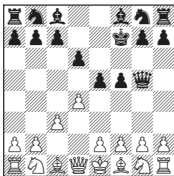


S3

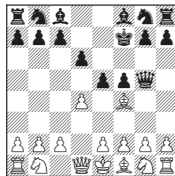
A critic scores each option



s_1



s_2

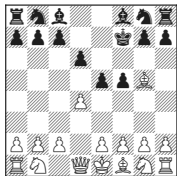


s_3

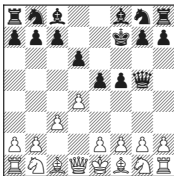
$$V(s_1) = +6.94$$

A chess engine's score, in pawns: +1.00 means "a pawn up". Stockfish 18 at depth 18.

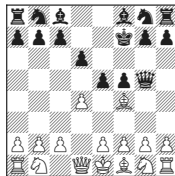
A critic scores each option



s_1



s_2

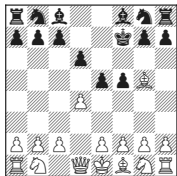


s_3

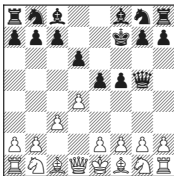
$$V(s_1) = +6.94 \quad V(s_2) = +0.30$$

A chess engine's score, in pawns: +1.00 means "a pawn up". Stockfish 18 at depth 18.

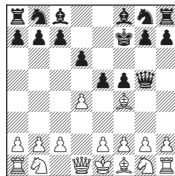
A critic scores each option



s_1



s_2



s_3

$$V(s_1) = +6.94$$

$$V(s_2) = +0.30$$

$$V(s_3) = -3.45$$

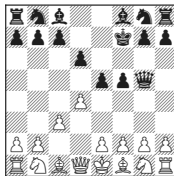
A chess engine's score, in pawns: +1.00 means "a pawn up". Stockfish 18 at depth 18.

A critic scores each option



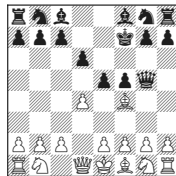
$$V(s_1) = +6.94$$

play this move



s_2

$$V(s_2) = +0.30$$



s_3

$$V(s_3) = -3.45$$

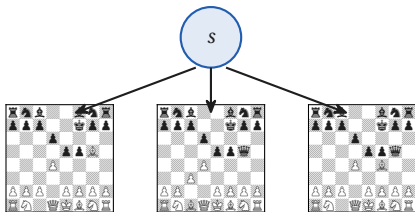
A chess engine's score, in pawns: +1.00 means "a pawn up". Stockfish 18 at depth 18.

Search with the critic alone



Repeat until the game ends:

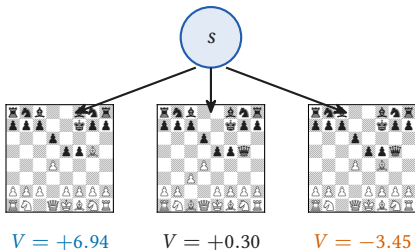
Search with the critic alone



Repeat until the game ends:

1. List the actions and the state each one leads to

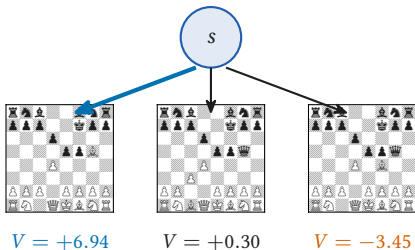
Search with the critic alone



Repeat until the game ends:

1. List the actions and the state each one leads to
2. Compute $V(s)$ for each

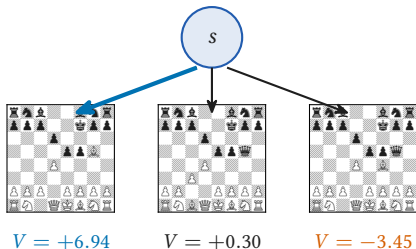
Search with the critic alone



Repeat until the game ends:

1. List the actions and the state each one leads to
2. Compute $V(s)$ for each
3. Play the action with the largest $V(s)$

Search with the critic alone



Repeat until the game ends:

1. List the actions and the state each one leads to
2. Compute $V(s)$ for each
3. Play the action with the largest $V(s)$
4. Continue from that state

No game is ever played out. The critic is asked once per option, and we move. This already plays chess, and on this position it plays the right move.

Why the critic alone is not enough

the move

a simple critic: count the material

what actually happens

take the queen

+9

a queen up

Why the critic alone is not enough

the move	a simple critic: count the material	what actually happens
take the queen	+9	a queen up
a quiet pawn move	0	level, nothing lost
stop one square short	0	a pawn eats the bishop next move: -3

Why the critic alone is not enough

the move	a simple critic: count the material	what actually happens
take the queen	+9	a queen up
a quiet pawn move	0	level, nothing lost
stop one square short	0	a pawn eats the bishop next move: -3

the critic gives these two the same score

Why the critic alone is not enough

the move	a simple critic: count the material	what actually happens
take the queen	+9	a queen up
a quiet pawn move	0	level, nothing lost
stop one square short	0	a pawn eats the bishop next move: -3

the critic gives these two the same score

- Everything rests on V : one wrong score and the search walks straight in

Why the critic alone is not enough

the move	a simple critic: count the material	what actually happens
take the queen	+9	a queen up
a quiet pawn move	0	level, nothing lost
stop one square short	0	a pawn eats the bishop next move: -3

the critic gives these two the same score

- Everything rests on V : one wrong score and the search walks straight in
- Nothing is ever played out, so no real outcome ever comes back to correct it

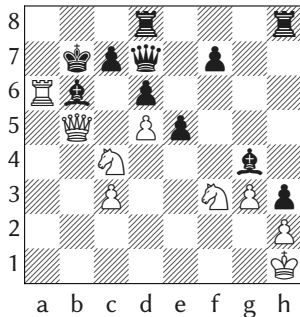
Why the critic alone is not enough

the move	a simple critic: count the material	what actually happens
take the queen	+9	a queen up
a quiet pawn move	0	level, nothing lost
stop one square short	0	a pawn eats the bishop next move: -3

the critic gives these two the same score

- Everything rests on V : one wrong score and the search walks straight in
- Nothing is ever played out, so no real outcome ever comes back to correct it
- V is an **estimate**, not the truth. We need to know how far off it might be.

A harder position: which move would you play?

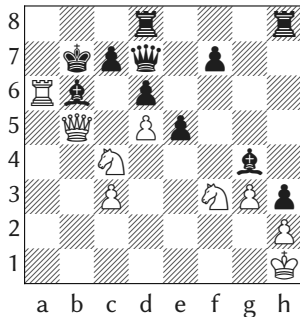


White to move

the move

what the material critic sees

A harder position: which move would you play?



White to move

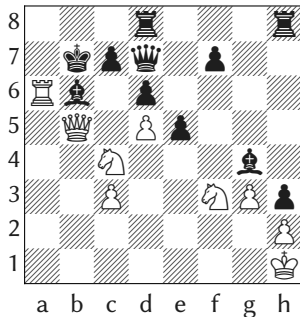
the move

what the material critic sees

Qxd7 takes the queen

+9

A harder position: which move would you play?



White to move

the move

what the material critic sees

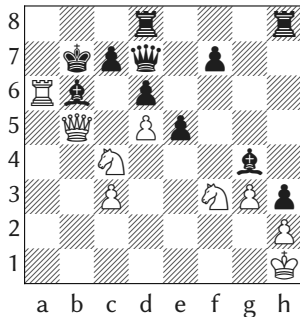
Qxd7 takes the queen

+9

Rxb6+ takes the bishop,
with check

+3

A harder position: which move would you play?



White to move

the move

what the material critic sees

Qxd7 takes the queen

+9

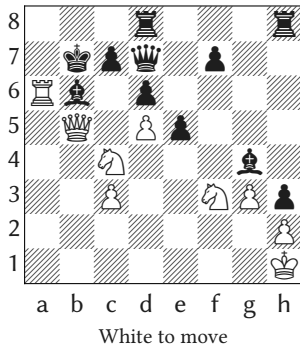
Rxb6+ takes the bishop,
with check

+3

Na5+ a check that hangs
the knight

0

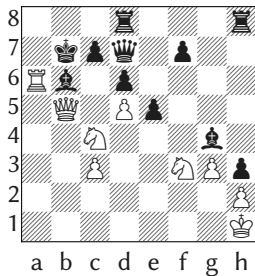
A harder position: which move would you play?



the move	what the material critic sees
Qxd7 takes the queen	+9
Rxb6+ takes the bishop, with check	+3
Na5+ a check that hangs the knight	0

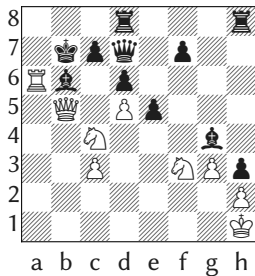
So the critic would play **Qxd7**: it wins a whole queen, and the other two win less or nothing.

What actually happens



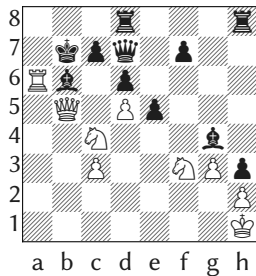
the move	critic	the truth
Qxd7	+9	mated in 1: 28...Bxf3#

What actually happens



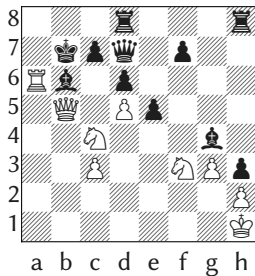
the move	critic	the truth
Qxd7	+9	mated in 1: 28...Bxf3#
Rxb6+	+3	-5.5: still losing

What actually happens



the move	critic	the truth
Qxd7	+9	mated in 1: 28...Bxf3#
Rxb6+	+3	−5.5: still losing
Na5+	0	mate in 4, forced: 28...Kb8 29.Ra8+ Kxa8 30.Qa6+ Kb8 31.Qb7#

What actually happens



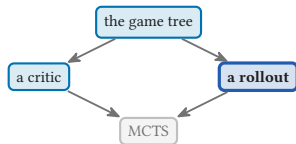
the move	critic	the truth
Qxd7	+9	mated in 1: 28...Bxf3#
Rxb6+	+3	-5.5: still losing
Na5+	0	mate in 4, forced: 28...Kb8 29.Ra8+ Kxa8 30.Qa6+ Kb8 31.Qb7#

The critic ranked them Qxd7, Rxb6+, Na5+. The truth is **exactly the reverse**. Taking the queen loses on the spot; giving a knight away wins.



Section 5

Scoring a state with a rollout



5

Different options for V

A domain critic

a hand-built scoring
function, like a
chess engine's

Different options for V

A domain critic

a hand-built scoring
function, like a
chess engine's

somebody has to write it,
and you just watched it
rank three moves backwards

Different options for V

A domain critic

a hand-built scoring
function, like a
chess engine's

Run rollouts

play it out from
here many times,
and count how
often you win

somebody has to write it,
and you just watched it
rank three moves backwards

Different options for V

A domain critic

a hand-built scoring
function, like a
chess engine's

somebody has to write it,
and you just watched it
rank three moves backwards

Run rollouts

play it out from
here many times,
and count how
often you win

works in any domain you
can simulate, but each
estimate costs many games

Different options for V

A domain critic

a hand-built scoring
function, like a
chess engine's

somebody has to write it,
and you just watched it
rank three moves backwards

Run rollouts

play it out from
here many times,
and count how
often you win

works in any domain you
can simulate, but each
estimate costs many games

Ask a language model

show it the state
and ask how
promising it looks

Different options for V

A domain critic

a hand-built scoring function, like a chess engine's

somebody has to write it,
and you just watched it
rank three moves backwards

Run rollouts

play it out from
here many times,
and count how
often you win

works in any domain you
can simulate, but each
estimate costs many games

Ask a language model

show it the state
and ask how
promising it looks

works where nothing
can be simulated, but the
score is an opinion (this
one is the 10:00 lecture)

Different options for V

A domain critic

a hand-built scoring
function, like a
chess engine's

somebody has to write it,
and you just watched it
rank three moves backwards

Run rollouts

play it out from
here many times,
and count how
often you win

works in any domain you
can simulate, but each
estimate costs many games

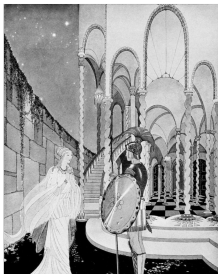
Ask a language model

show it the state
and ask how
promising it looks

works where nothing
can be simulated, but the
score is an opinion (this
one is the 10:00 lecture)

The middle one needs no domain knowledge and no expert, so that is the one this section builds. It is also a very old idea. Older than chess engines.

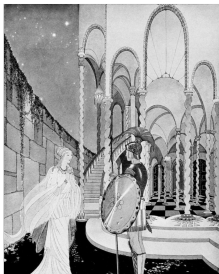
The Minotaur, and the two sails



Virginia Frances Sterrett,
Tanglewood Tales, 1921.

On Crete, King Minos kept a monster: half man and half bull, in a labyrinth Daedalus built so well he could barely escape it.

The Minotaur, and the two sails

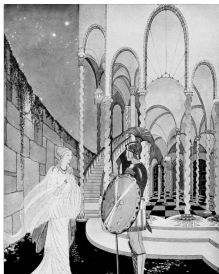


Virginia Frances Sterrett,
Tanglewood Tales, 1921.

On Crete, King Minos kept a monster: half man and half bull, in a labyrinth Daedalus built so well he could barely escape it.

Athens owed Minos a tribute: seven young men and seven young women, sent in to be devoured. None came out. One year Theseus, the king's son, went in with them.

The Minotaur, and the two sails



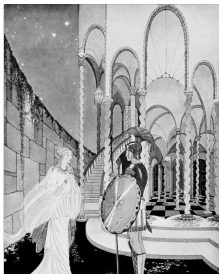
Virginia Frances Sterrett,
Tanglewood Tales, 1921.

On Crete, King Minos kept a monster: half man and half bull, in a labyrinth Daedalus built so well he could barely escape it.

Athens owed Minos a tribute: seven young men and seven young women, sent in to be devoured. None came out. One year Theseus, the king's son, went in with them.

The ships to Crete sailed under a black sail, for mourning. His father Aegeus asked one thing of him: if you live, come home under a white one.

The Minotaur, and the two sails



Virginia Frances Sterrett,
Tanglewood Tales, 1921.

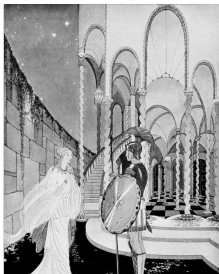
On Crete, King Minos kept a monster: half man and half bull, in a labyrinth Daedalus built so well he could barely escape it.

Athens owed Minos a tribute: seven young men and seven young women, sent in to be devoured. None came out. One year Theseus, the king's son, went in with them.

The ships to Crete sailed under a black sail, for mourning. His father Aegeus asked one thing of him: if you live, come home under a white one.

He would be watching from the cliff, too far off to see anything but the colour of the sail.

The Minotaur, and the two sails



Virginia Frances Sterrett,
Tanglewood Tales, 1921.

On Crete, King Minos kept a monster: half man and half bull, in a labyrinth Daedalus built so well he could barely escape it.

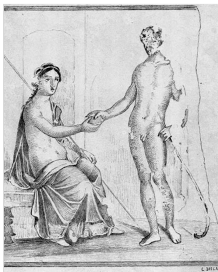
Athens owed Minos a tribute: seven young men and seven young women, sent in to be devoured. None came out. One year Theseus, the king's son, went in with them.

The ships to Crete sailed under a black sail, for mourning. His father Aegeus asked one thing of him: if you live, come home under a white one.

He would be watching from the cliff, too far off to see anything but the colour of the sail.

Apollodorus, *Epitome*, trans. J. G. Frazer, 1921.

The thread, and the black sail



Ariadne hands Theseus the thread. Pompeii, 1st c.

Ariadne, the king's daughter, put the right question to Daedalus. Not how to kill the monster: how to get out again. He told her: a thread.

The thread, and the black sail



Ariadne hands Theseus the thread. Pompeii, 1st c.

Ariadne, the king's daughter, put the right question to Daedalus. Not how to kill the monster: how to get out again. He told her: a thread.

Theseus tied one end at the door, walked in, killed the Minotaur, and followed the thread back out into the daylight.

The thread, and the black sail



Ariadne hands Theseus the thread. Pompeii, 1st c.

Ariadne, the king's daughter, put the right question to Daedalus. Not how to kill the monster: how to get out again. He told her: a thread.

Theseus tied one end at the door, walked in, killed the Minotaur, and followed the thread back out into the daylight.

He had done what nobody had done. Then he sailed for home, and forgot the sail.

The thread, and the black sail



Ariadne hands Theseus the thread. Pompeii, 1st c.

Ariadne, the king's daughter, put the right question to Daedalus. Not how to kill the monster: how to get out again. He told her: a thread.

Theseus tied one end at the door, walked in, killed the Minotaur, and followed the thread back out into the daylight.

He had done what nobody had done. Then he sailed for home, and forgot the sail.

Aegeus, watching from the cliff, saw **black**, read it as he had agreed to read it, and threw himself into the sea.

The thread, and the black sail



Ariadne hands Theseus the thread. Pompeii, 1st c.

Ariadne, the king's daughter, put the right question to Daedalus. Not how to kill the monster: how to get out again. He told her: a thread.

Theseus tied one end at the door, walked in, killed the Minotaur, and followed the thread back out into the daylight.

He had done what nobody had done. Then he sailed for home, and forgot the sail.

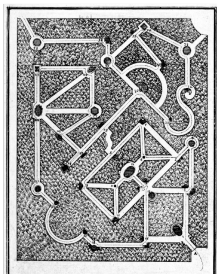
Aegeus, watching from the cliff, saw **black**, read it as he had agreed to read it, and threw himself into the sea.

Apollodorus, *Epitome* 1.8 to 1.11, trans. J. G. Frazer, 1921.

ArCADE Th



Theseus and Ariadne: the playout



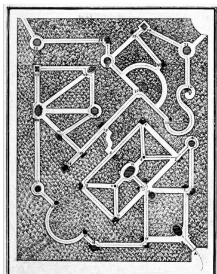
Plan of the labyrinth of Versailles, 1677.



One line, played to the end

- One line, not the whole maze: the thread is a sample, and it is the only one they take.

Theseus and Ariadne: the payout



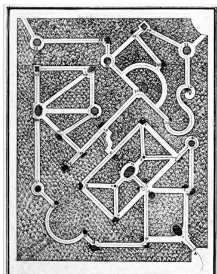
Plan of the labyrinth of Versailles, 1677.



One line, played to the end

- One line, not the whole maze: the thread is a sample, and it is the only one they take.
- The same thread carries him home, so the result of that one line reaches the door.

Theseus and Ariadne: the playout



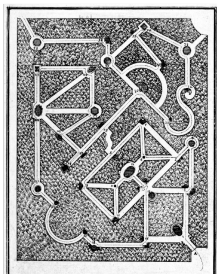
Plan of the labyrinth of Versailles, 1677.



One line, played to the end

- One line, not the whole maze: the thread is a sample, and it is the only one they take.
- The same thread carries him home, so the result of that one line reaches the door.
- **In the search:** one line played to a finished game, and the result carried back up the path.

Theseus and Ariadne: the playout



Plan of the labyrinth of Versailles, 1677.



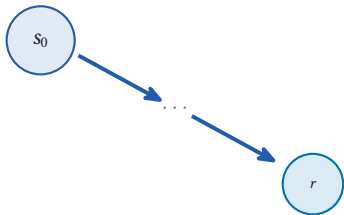
One line, played to the end

- One line, not the whole maze: the thread is a sample, and it is the only one they take.
- The same thread carries him home, so the result of that one line reaches the door.
- **In the search:** one line played to a finished game, and the result carried back up the path.
- **Their flaw:** the black sail. The result came home right and was read wrong, and his father died of it.

What a rollout gives us

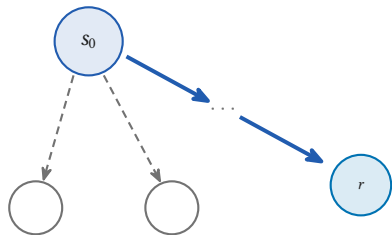


What a rollout gives us



- The reward is **real and measured**. No critic, no domain knowledge, nothing to hand-build

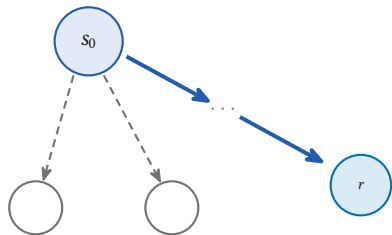
What a rollout gives us



the moves it never tried

- The reward is **real and measured**. No critic, no domain knowledge, nothing to hand-build
- But it followed **one line** out of many that exist from s_0

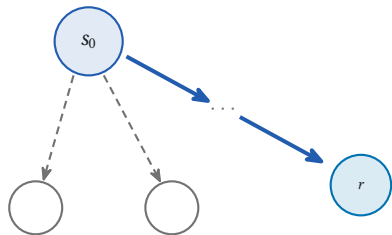
What a rollout gives us



the moves it never tried

- The reward is **real and measured**. No critic, no domain knowledge, nothing to hand-build
- But it followed **one line** out of many that exist from s_0
- Run it again and the number can come back different: one playout is one **sample** of how games from here end

What a rollout gives us



the moves it never tried

- The reward is **real and measured**. No critic, no domain knowledge, nothing to hand-build
- But it followed **one line** out of many that exist from s_0
- Run it again and the number can come back different: one playout is one **sample** of how games from here end

So run many from the same state and average them. That average is our $V(s)$, and Ariadne's thread is what carries each result home.

Three moves, one hundred playouts each

X	X	.
O	O	.
X	O	.

X to move

Three moves, one hundred playouts each

X	X	.
O	O	.
X	O	.

X to move

no critic this time: we buy each score
with playouts

X	X	X
O	O	.
X	O	.

X	X	.
O	O	X
X	O	.

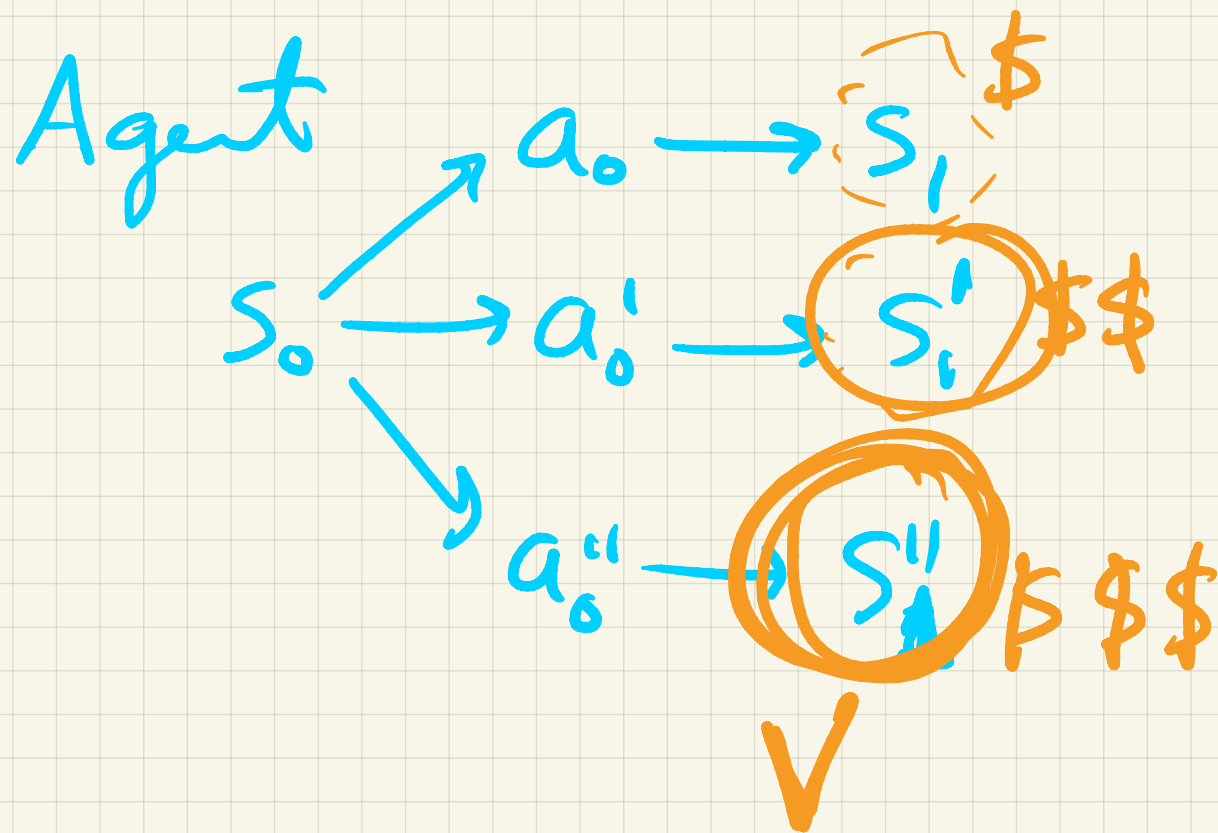
X	X	.
O	O	.
X	O	X

1) Thinking agents w/ RL

↳ Agent

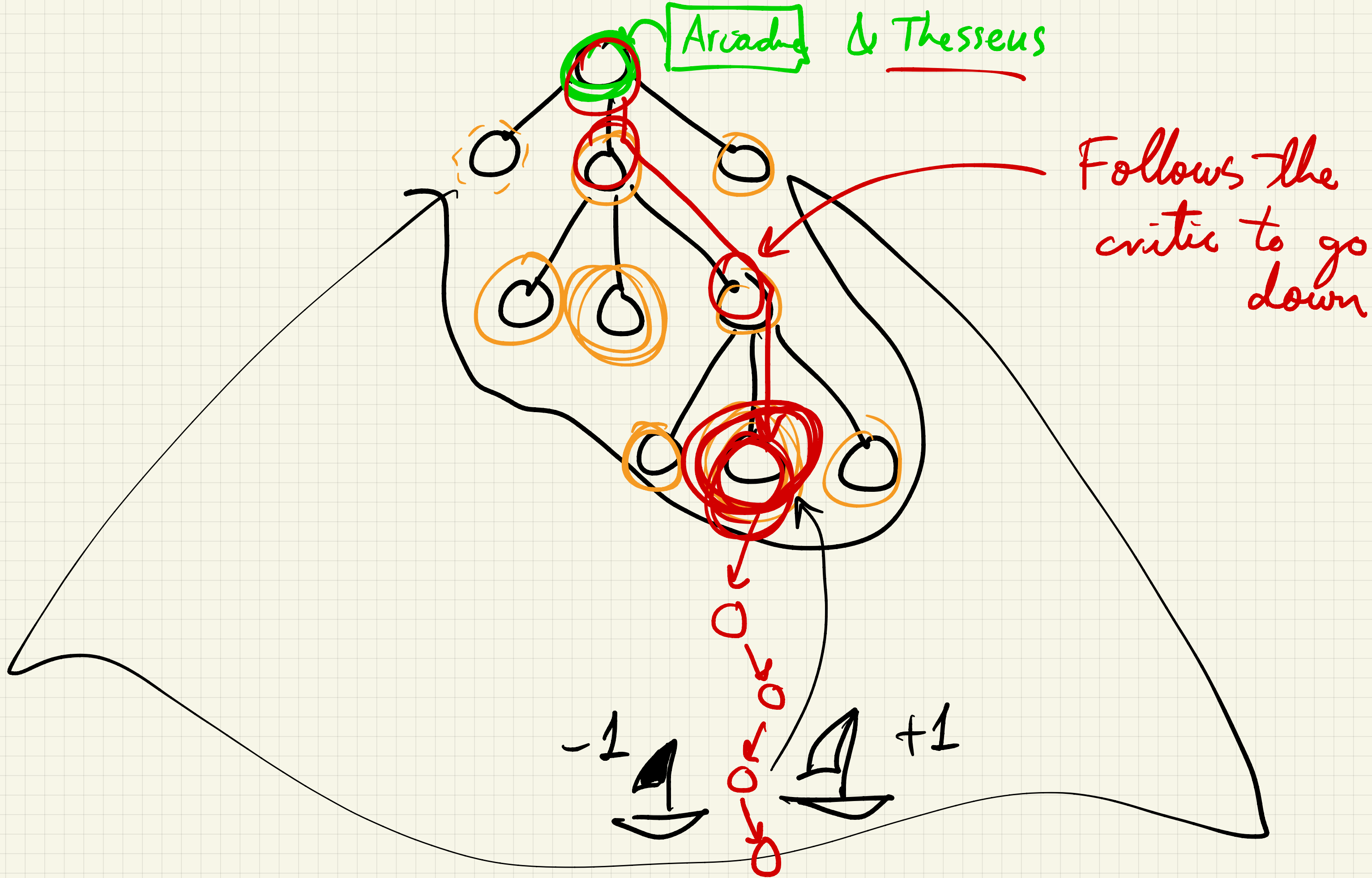
$s_0 \rightarrow a_0 \rightarrow r_0 \rightarrow s_1 \rightarrow a_1 \rightarrow r_1 \rightarrow s_2 \dots$

2) Think agents w/ Monte-Carlo Tree search



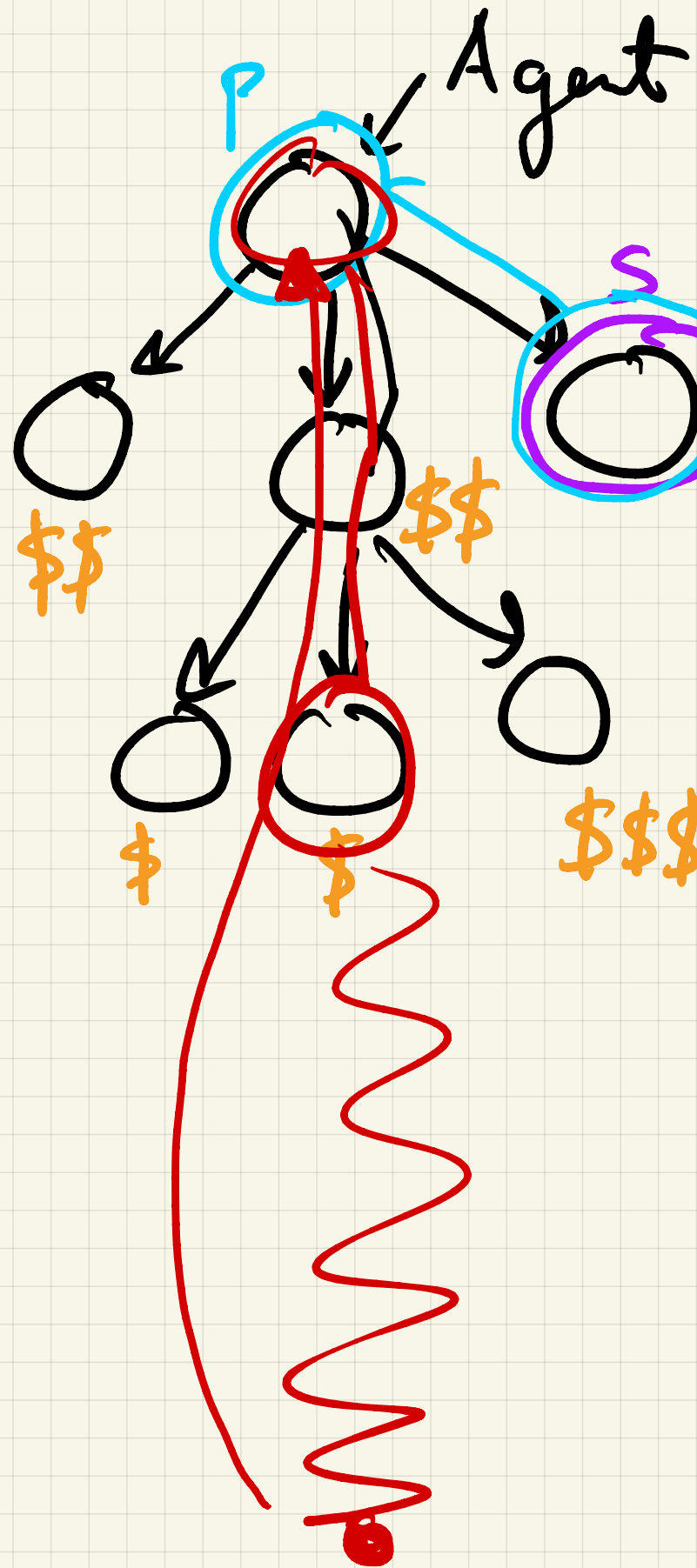
& Thesens

Follows the
critic to go
down



MCTS

Thesaur I have an agent (LLM)



Critic

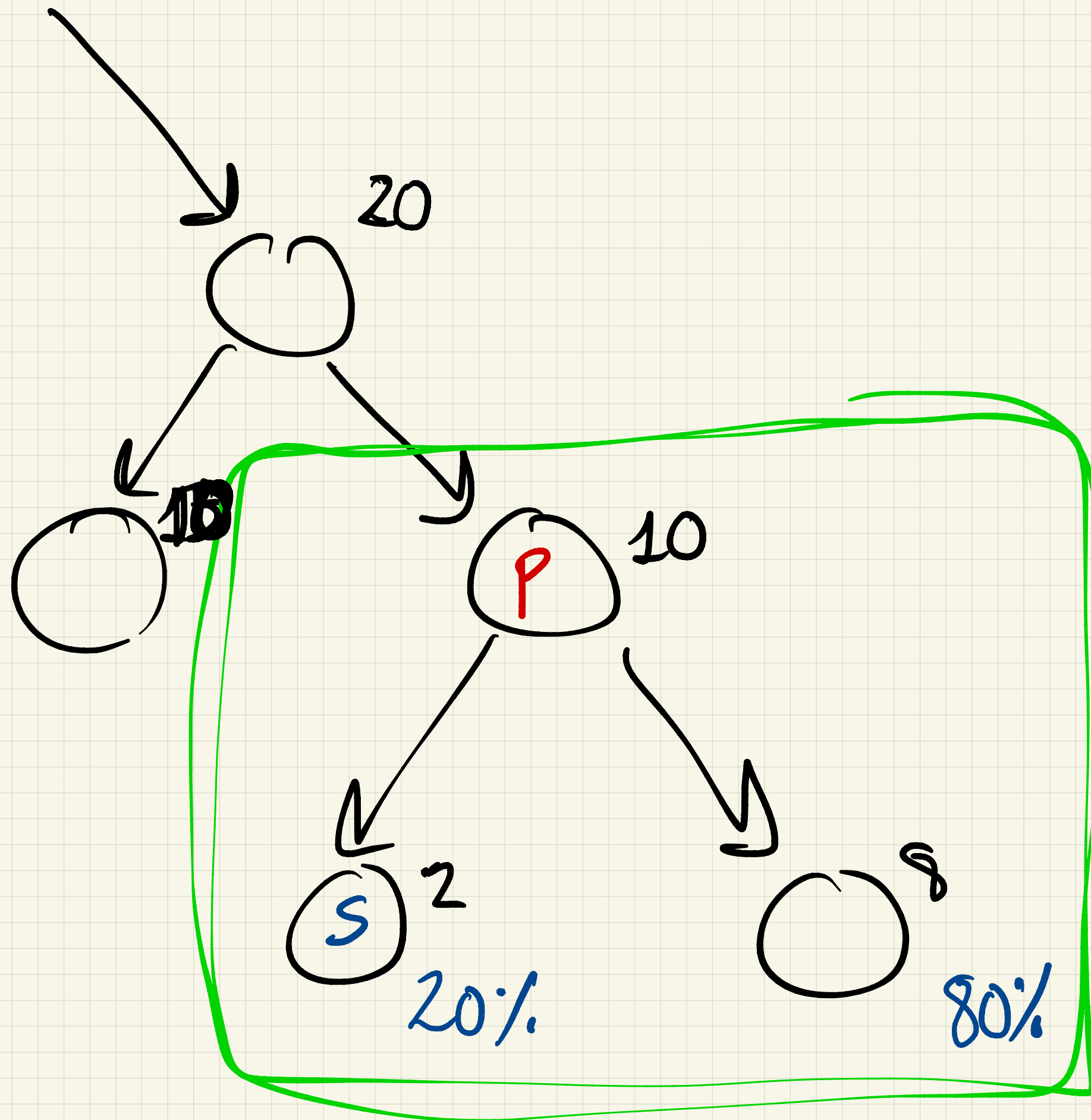
is not perfect !!

how "unexplored" this part of the tree is

$$UCT(s) = V(s) + w \sqrt{\frac{\ln N(p)}{N(s)}}$$

of times visited s

of times Thesaur visited

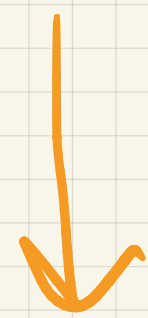
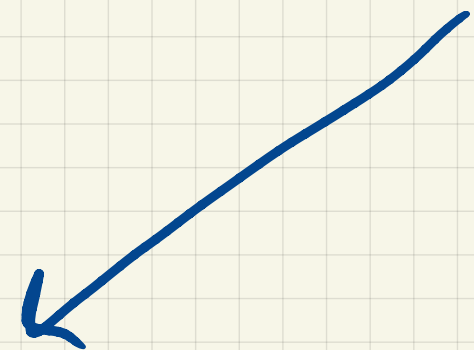


1st attempt

$$\frac{N(s)}{N(p)} \rightarrow \sqrt{\frac{N(p)}{N(s)}}$$

$$\sqrt{\frac{\log N(p)}{N(s)}}$$

X		X
X	O	O
O	X	



X	X	X
X	O	O
O	X	

X		X
X	O	O
O	X	X

Three moves, one hundred playouts each

X	X	.
O	O	.
X	O	.

X to move

no critic this time: we buy each score
with playouts

X	X	X
O	O	.
X	O	.

1.00

every playout won

X	X	.
O	O	X
X	O	.

X	X	.
O	O	.
X	O	X

Three moves, one hundred playouts each

X	X	.
O	O	.
X	O	.

X to move

no critic this time: we buy each score
with playouts

X	X	X
O	O	.
X	O	.

1.00

every playout won

X	X	.
O	O	X
X	O	.

0.77

random O often slips

X	X	.
O	O	.
X	O	X

Three moves, one hundred playouts each

X	X	.
O	O	.
X	O	.

X to move

no critic this time: we buy each score
with playouts

X	X	X
O	O	.
X	O	.

1.00

every playout won

X	X	.
O	O	X
X	O	.

0.77

random O often slips

X	X	.
O	O	.
X	O	X

0.23

lost, most of the time

Three moves, one hundred playouts each

X	X	.
O	O	.
X	O	.

X to move

no critic this time: we buy each score
with playouts

X	X	X
O	O	.
X	O	.

1.00

every playout won

X	X	.
O	O	X
X	O	.

0.77

random O often slips

X	X	.
O	O	.
X	O	X

0.23

lost, most of the time

The ranking is right, and the numbers are not the truth: the middle move is a draw under best play, not 0.77. A playout measures [random](#) play, and here that is enough to sort the moves.

100 uniformly random playouts per move, seed 0, reward 1 win, 0.5 draw, 0 loss: `reference_slides.py`

The same trick on the chess position

rollout average, 400 playouts, seed 7

the critic, section 4

Bxg5, takes the queen



0.52

+6.94

The same trick on the chess position

	rollout average, 400 playouts, seed 7	the critic, section 4
Bxg5, takes the queen	 0.52	+6.94
c3, quiet	 0.51	+0.30

The same trick on the chess position

	rollout average, 400 playouts, seed 7	the critic, section 4
Bxg5, takes the queen	 0.52	+6.94
c3, quiet	 0.51	+0.30
Bf4, the blunder	 0.52	-3.45

The same trick on the chess position

	rollout average, 400 playouts, seed 7	the critic, section 4
Bxg5, takes the queen	 0.52	+6.94
c3, quiet	 0.51	+0.30
Bf4, the blunder	 0.52	-3.45

- The three averages are the same number

The same trick on the chess position

	rollout average, 400 playouts, seed 7	the critic, section 4
Bxg5, takes the queen	 0.52	+6.94
c3, quiet	 0.51	+0.30
Bf4, the blunder	 0.52	-3.45

- The three averages are the **same number**
- An extra queen is worth nothing to a player who moves at random

The same trick on the chess position

	rollout average, 400 playouts, seed 7	the critic, section 4
Bxg5, takes the queen	 0.52	+6.94
c3, quiet	 0.51	+0.30
Bf4, the blunder	 0.52	-3.45

- The three averages are the **same number**
- An extra queen is worth nothing to a player who moves at random
- A rollout measures what random play can reach: in chess, almost nothing

The same trick on the chess position

	rollout average, 400 playouts, seed 7	the critic, section 4
Bxg5, takes the queen	 0.52	+6.94
c3, quiet	 0.51	+0.30
Bf4, the blunder	 0.52	-3.45

- The three averages are the **same number**
- An extra queen is worth nothing to a player who moves at random
- A rollout measures what random play can reach: in chess, almost nothing

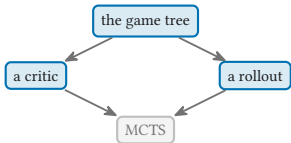
Same reason AlphaGo did not play out at random: a small trained policy drove its playouts (Silver et al. 2016).



Section 6

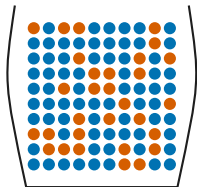
How much to believe one number

6



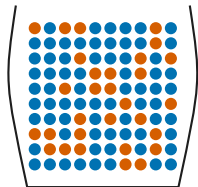
one more tool before the loop

One urn, three sample sizes



70 wins, 30 losses,
so the truth is 0.70

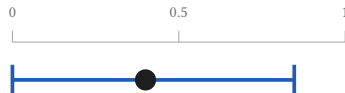
One urn, three sample sizes



take 5

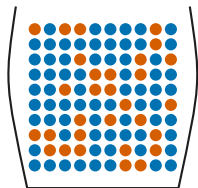


$$\hat{V} = 0.40$$



70 wins, 30 losses,
so the truth is 0.70

One urn, three sample sizes



take 5



take 20



70 wins, 30 losses,
so the truth is 0.70

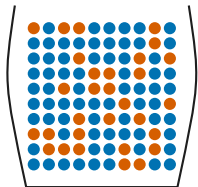
$$\hat{V} = 0.40$$



$$\hat{V} = 0.75$$



One urn, three sample sizes



70 wins, 30 losses,
so the truth is 0.70

take 5



$$\hat{V} = 0.40$$



take 20



$$\hat{V} = 0.75$$



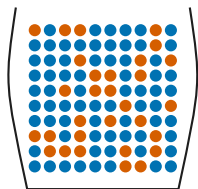
take 90



$$\hat{V} = 0.689$$



One urn, three sample sizes



70 wins, 30 losses,
so the truth is 0.70

take 5



$$\hat{V} = 0.40$$

take 20

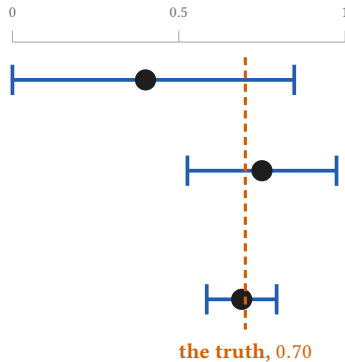


$$\hat{V} = 0.75$$

take 90

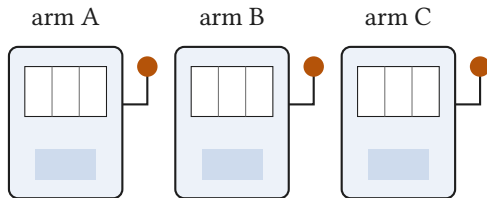


$$\hat{V} = 0.689$$



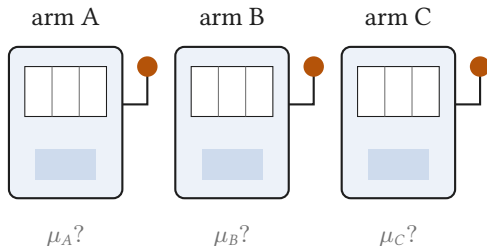
The interval is $\hat{V} \pm 1/\sqrt{n}$. The estimate wobbles (0.40, then 0.75, then 0.689); it is the **interval** that shrinks, and all three contain the truth.

Three bandits



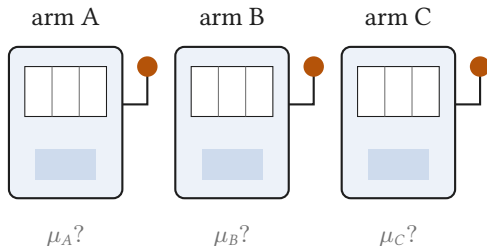
- Three arms, one pull at a time, a fixed number of pulls in total

Three bandits



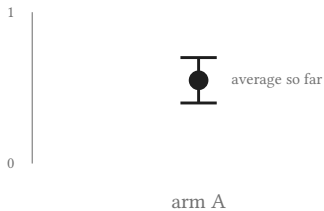
- Three arms, one pull at a time, a fixed number of pulls in total
- Arm i pays out around a **hidden** average μ_i , which you never see and can only estimate by pulling

Three bandits

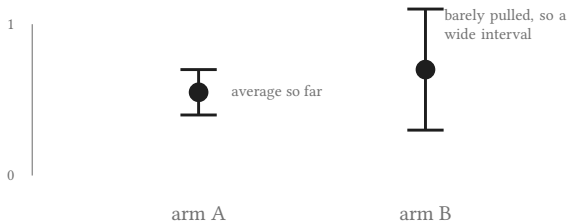


- Three arms, one pull at a time, a fixed number of pulls in total
- Arm i pays out around a **hidden** average μ_i , which you never see and can only estimate by pulling
- Each of our three chess moves is an arm, and a pull is one game played out from it

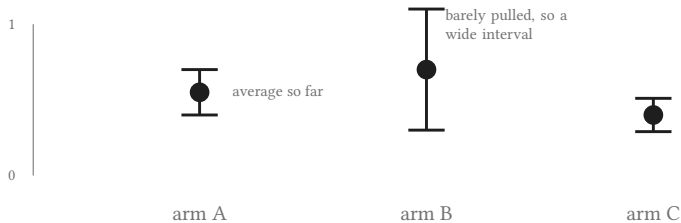
Pick the arm with the highest upper bound



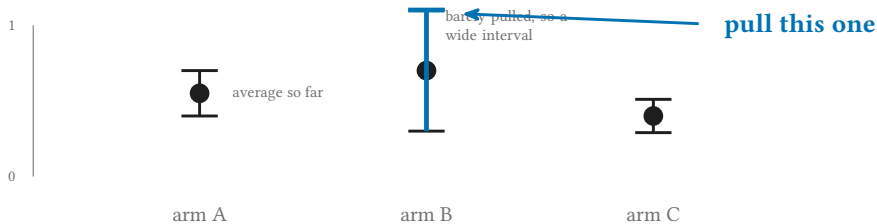
Pick the arm with the highest upper bound



Pick the arm with the highest upper bound

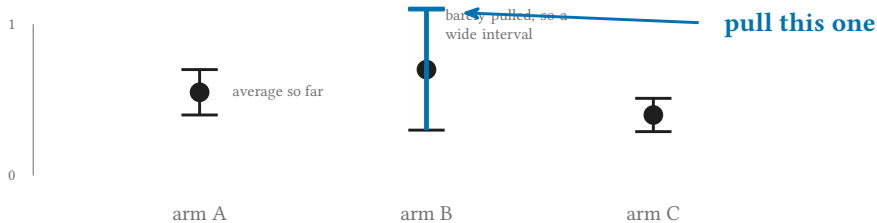


Pick the arm with the highest upper bound



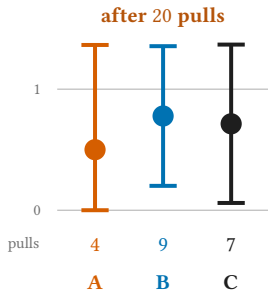
- Not the highest **average**: the highest **top of the interval**

Pick the arm with the highest upper bound

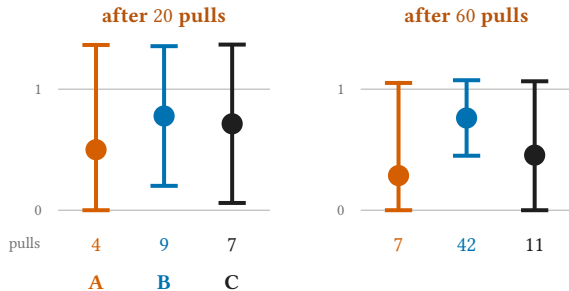


- Not the highest **average**: the highest **top of the interval**
- A barely pulled arm has a wide interval, so it gets tried even when its average is poor. Pull it and the interval shrinks.

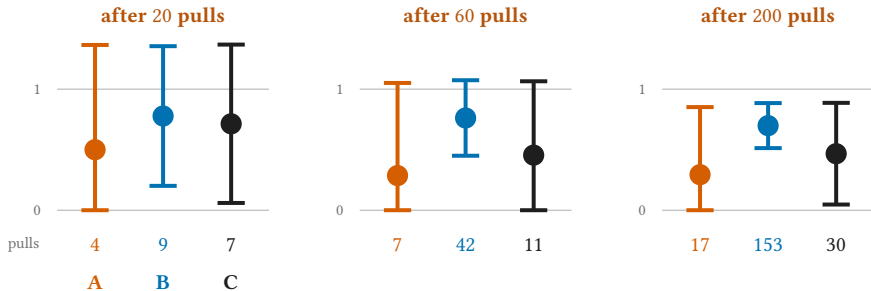
Watch it run: 200 pulls



Watch it run: 200 pulls

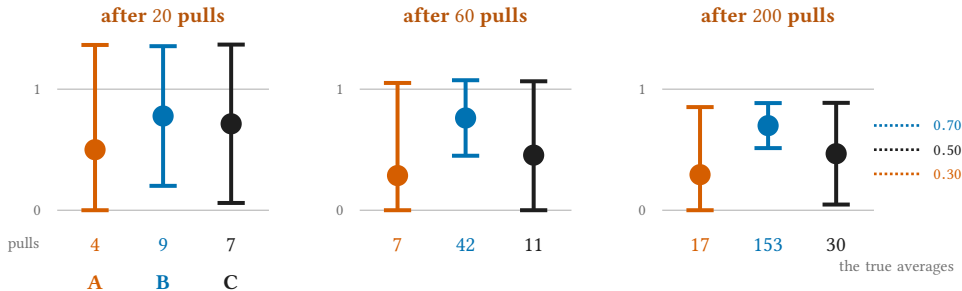


Watch it run: 200 pulls



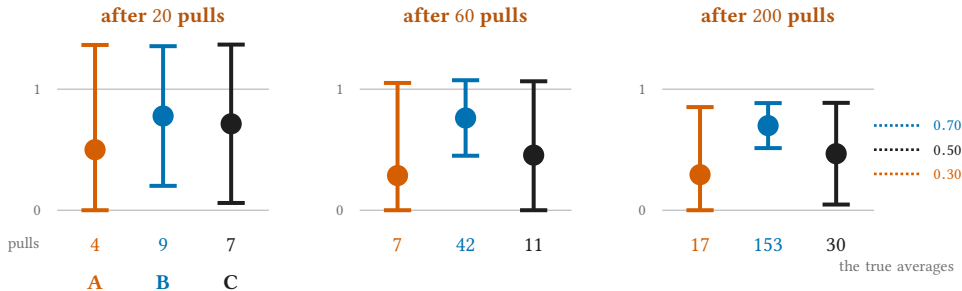
- B's interval collapses; A's and C's stay wide, because only B was pulled enough to pin down

Watch it run: 200 pulls



- B's interval collapses; A's and C's stay wide, because only B was pulled enough to pin down
- The budget went almost entirely to B: 153 of 200 pulls

Watch it run: 200 pulls

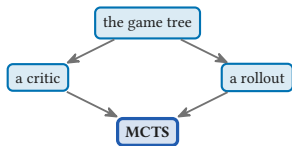


- B's interval collapses; A's and C's stay wide, because only B was pulled enough to pin down
- The budget went almost entirely to B: 153 of 200 pulls
- The **tops stay level**: UCB never abandons an arm, it just visits the poor ones more and more rarely



Section 7

Monte Carlo Tree Search



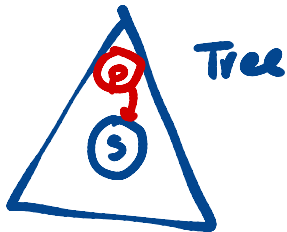
UCT: the score we actually use

$$\text{UCT}(\underline{s}) = \underbrace{V(s)}_{\text{how good it looked}} +$$

$$\underbrace{w \sqrt{\frac{\ln N(p)}{N(s)}}}_{\text{how little we know}}$$

Handwritten annotations:

- You** (red) with a red arrow pointing to the w term.
- # times** (green) with a green arrow pointing to $\ln N(p)$.
- Thesseus visited p** (green) with a green arrow pointing to $N(p)$.
- Thesseus visited s** (blue) with a blue arrow pointing to $N(s)$.



UCT: the score we actually use

$$\text{UCT}(s) = \underbrace{V(s)}_{\text{how good it looked}} + \underbrace{w \sqrt{\frac{\ln N(p)}{N(s)}}}_{\text{how little we know}}$$

$V(s)$: The average reward we have seen from s

UCT: the score we actually use

$$\text{UCT}(s) = \underbrace{V(s)}_{\text{how good it looked}} + \underbrace{w \sqrt{\frac{\ln N(p)}{N(s)}}}_{\text{how little we know}}$$

$V(s)$: The average reward we have seen from s

$N(s)$: How many times we tried s

UCT: the score we actually use

$$\text{UCT}(s) = \underbrace{V(s)}_{\text{how good it looked}} + \underbrace{w \sqrt{\frac{\ln N(p)}{N(s)}}}_{\text{how little we know}}$$

$V(s)$: The average reward we have seen from s

$N(s)$: How many times we tried s

$N(p)$: How many tries in total

UCT: the score we actually use

$$\text{UCT}(s) = \underbrace{V(s)}_{\text{how good it looked}} + \underbrace{w \sqrt{\frac{\ln N(p)}{N(s)}}}_{\text{how little we know}}$$

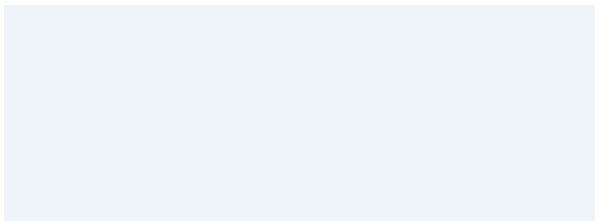
$V(s)$: The average reward we have seen from s

$N(s)$: How many times we tried s

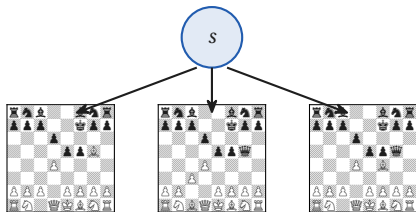
$N(p)$: How many tries in total

w : How strongly we favour the unknown; we use $w = 1$

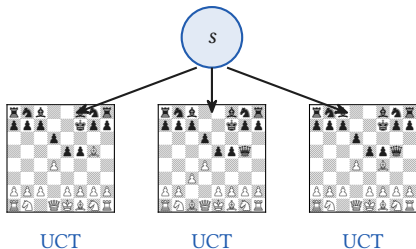
One run of the search



One run of the search

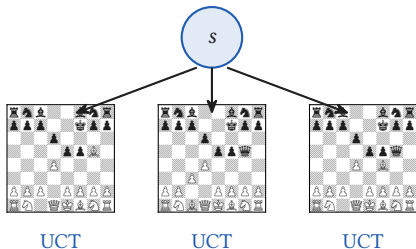


One run of the search



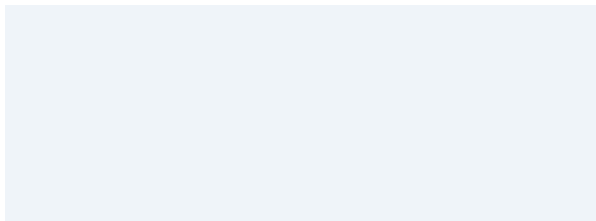
one score per child, recomputed every time round

One run of the search

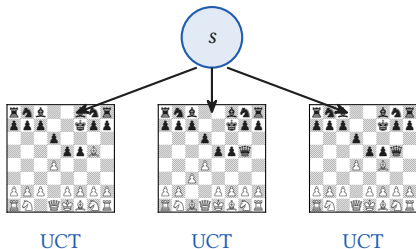


one score per child, recomputed every time round

Repeat until the game ends:



One run of the search



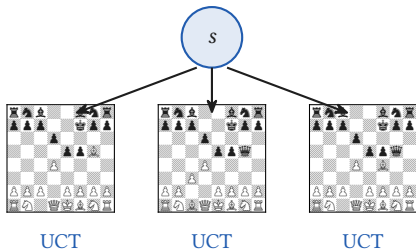
one score per child, recomputed every time round

Repeat until the game ends:

Repeat M times:

Planning; nothing real happens

One run of the search



one score per child, recomputed every time round

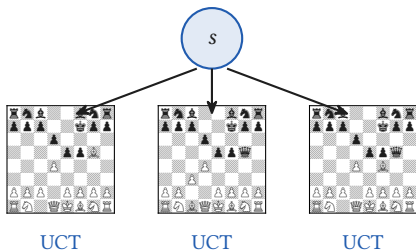
Repeat until the game ends:

Repeat M times:

Planning; nothing real happens

1. Compute UCT for each child

One run of the search



one score per child, recomputed every time round

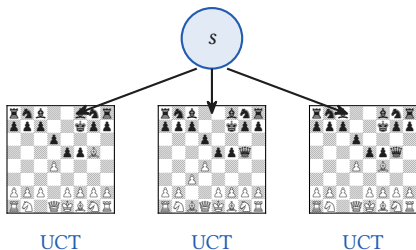
Repeat until the game ends:

Repeat M times:

Planning; nothing real happens

1. Compute UCT for each child
2. Descend into the highest

One run of the search



one score per child, recomputed every time round

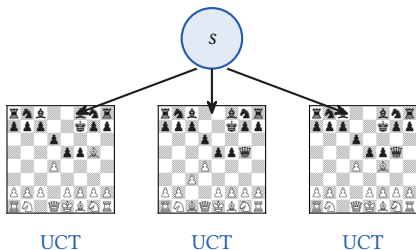
Repeat until the game ends:

Repeat M times:

Planning; nothing real happens

1. Compute UCT for each child
2. Descend into the highest
3. **do a rollout**: simulate one play to the end

One run of the search



one score per child, recomputed every time round

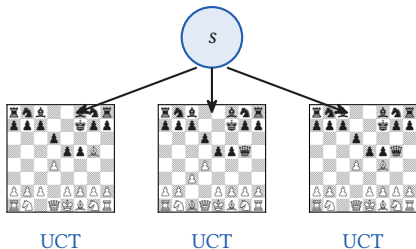
Repeat until the game ends:

Repeat M times:

Planning; nothing real happens

1. Compute UCT for each child
2. Descend into the highest
3. **do a rollout**: simulate one play to the end
4. Measure the reward

One run of the search



one score per child, recomputed every time round

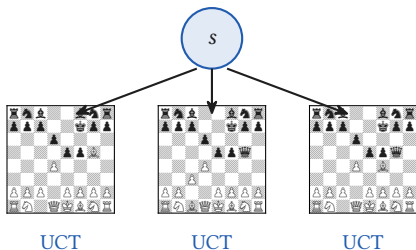
Repeat until the game ends:

Repeat M times:

Planning; nothing real happens

1. Compute UCT for each child
2. Descend into the highest
3. **do a rollout**: simulate one play to the end
4. Measure the reward
5. Update $V(s)$

One run of the search



one score per child, recomputed every time round

Repeat until the game ends:

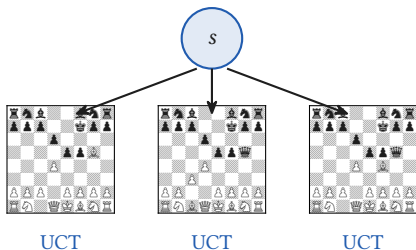
Repeat M times:

Planning; nothing real happens

1. Compute UCT for each child
2. Descend into the highest
3. **do a rollout**: simulate one play to the end
4. Measure the reward
5. Update $V(s)$

play the action with the largest $V(s)$

One run of the search



one score per child, recomputed every time round

Repeat until the game ends:

Repeat M times:

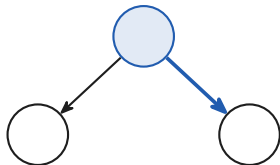
Planning; nothing real happens

1. Compute UCT for each child
2. Descend into the highest
3. **do a rollout**: simulate one play to the end
4. Measure the reward
5. Update $V(s)$

play the action with the largest $V(s)$
the state you land in becomes the new root

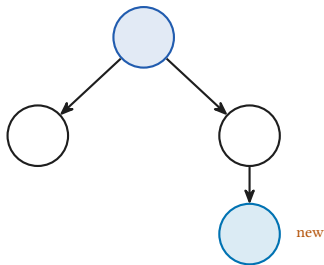
Only those last two lines happen in the real game. Everything in the box is imagined.

The four phases, precisely (optional)



1 Selection From the root, descend by UCT **while** the node is fully expanded and not terminal. Record the path.

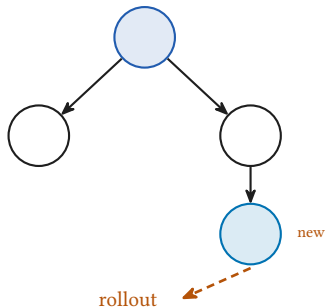
The four phases, precisely (optional)



1 Selection From the root, descend by UCT **while** the node is fully expanded and not terminal. Record the path.

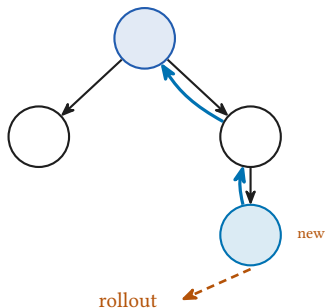
2 Expansion If that last node is not terminal, add **one** child, for its first untried action. It starts at $N = 1$, $V = 0$.

The four phases, precisely (optional)



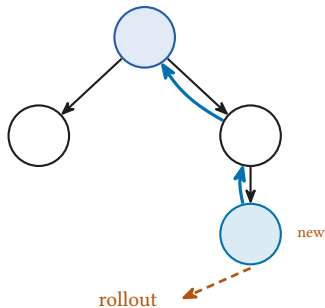
- 1 Selection** From the root, descend by UCT **while** the node is fully expanded and not terminal. Record the path.
- 2 Expansion** If that last node is not terminal, add **one** child, for its first untried action. It starts at $N = 1$, $V = 0$.
- 3 Simulation** Roll out from there until terminal or the step budget, and take the return r .

The four phases, precisely (optional)



- 1 Selection** From the root, descend by UCT **while** the node is fully expanded and not terminal. Record the path.
- 2 Expansion** If that last node is not terminal, add **one** child, for its first untried action. It starts at $N = 1$, $V = 0$.
- 3 Simulation** Roll out from there until terminal or the step budget, and take the return r .
- 4 Backprop** Walk the **whole recorded path** back to the root, updating every node on it.

The four phases, precisely (optional)



- 1 Selection** From the root, descend by UCT **while** the node is fully expanded and not terminal. Record the path.
- 2 Expansion** If that last node is not terminal, add **one** child, for its first untried action. It starts at $N = 1$, $V = 0$.
- 3 Simulation** Roll out from there until terminal or the step budget, and take the return r .
- 4 Backprop** Walk the **whole recorded path** back to the root, updating every node on it.

$$N(s) \leftarrow N(s) + 1$$
$$V(s) \leftarrow \frac{V(s) (N(s) - 1) + r}{N(s)}$$

One iteration adds at most one node.

The algorithm in full (optional)

The root starts at $N = 1, V = 0$

Repeat I times:

1. **Selection.** $path \leftarrow$ descend from the root, taking $\arg \max_c [V(c) + w\sqrt{\ln N(\text{node})/N(c)}]$, while the node is fully expanded and not terminal
2. **Expansion.** if the last node of $path$ is not terminal, add one child for its first untried action ($N = 1, V = 0$) and append it to $path$
3. **Simulation.** $r \leftarrow$ rollout from that node until terminal or the step budget
4. **Backpropagation.** for every node s in $path$: $N(s) \leftarrow N(s) + 1$, then $V(s) \leftarrow (V(s)(N(s) - 1) + r)/N(s)$

Return the child of the root with the largest visit count N

The algorithm in full (optional)

The root starts at $N = 1, V = 0$

Repeat I times:

1. **Selection.** $path \leftarrow$ descend from the root, taking $\arg \max_c [V(c) + w\sqrt{\ln N(\text{node})/N(c)}]$, while the node is fully expanded and not terminal
2. **Expansion.** if the last node of $path$ is not terminal, add one child for its first untried action ($N = 1, V = 0$) and append it to $path$
3. **Simulation.** $r \leftarrow$ rollout from that node until terminal or the step budget
4. **Backpropagation.** for every node s in $path$: $N(s) \leftarrow N(s) + 1$, then $V(s) \leftarrow (V(s)(N(s) - 1) + r)/N(s)$

Return the child of the root with the largest visit count N

- The loop two slides back left out [expansion](#): this is where new nodes come from, at most one per iteration

The algorithm in full (optional)

The root starts at $N = 1, V = 0$

Repeat I times:

1. **Selection.** $path \leftarrow$ descend from the root, taking $\arg \max_c [V(c) + w\sqrt{\ln N(\text{node})/N(c)}]$, while the node is fully expanded and not terminal
2. **Expansion.** if the last node of $path$ is not terminal, add one child for its first untried action ($N = 1, V = 0$) and append it to $path$
3. **Simulation.** $r \leftarrow$ rollout from that node until terminal or the step budget
4. **Backpropagation.** for every node s in $path$: $N(s) \leftarrow N(s) + 1$, then $V(s) \leftarrow (V(s)(N(s) - 1) + r)/N(s)$

Return the child of the root with the largest visit count N

- The loop two slides back left out **expansion**: this is where new nodes come from, at most one per iteration
- And it committed by largest V . The standard rule is largest N , a far less noisy signal. Both pick the same action on our run.

Worked: tic-tac-toe, three moves

X	X	.
O	O	.
X	O	.

X to move

Worked: tic-tac-toe, three moves

X	X	.
O	O	.
X	O	.

X to move

The three empty cells are the whole
right column

Worked: tic-tac-toe, three moves

X	X	.
O	O	.
X	O	.

X to move

The three empty cells are the whole
right column

X	X	X
O	O	.
X	O	.

X wins at once

Worked: tic-tac-toe, three moves

X	X	.
O	O	.
X	O	.

X to move

The three empty cells are the whole
right column

X	X	X
O	O	.
X	O	.

X wins at once

X	X	.
O	O	X
X	O	.

a draw

Worked: tic-tac-toe, three moves

X	X	.
O	O	.
X	O	.

X to move

The three empty cells are the whole
right column

X	X	X
O	O	.
X	O	.

X wins at once

X	X	.
O	O	X
X	O	.

a draw

X	X	.
O	O	.
X	O	X

X loses

O answers in the middle row and wins

Worked: tic-tac-toe, three moves

X	X	.
O	O	.
X	O	.

X to move

X	X	X
O	O	.
X	O	.

X wins at once

X	X	.
O	O	X
X	O	.

a draw

X	X	.
O	O	.
X	O	X

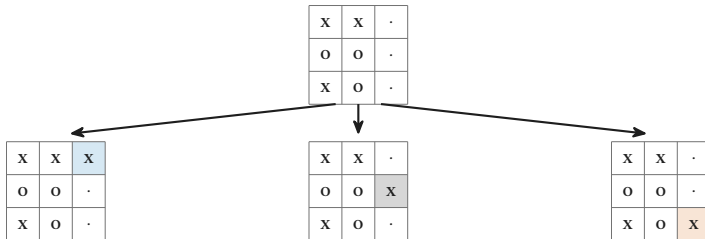
X loses

The three empty cells are the whole
right column

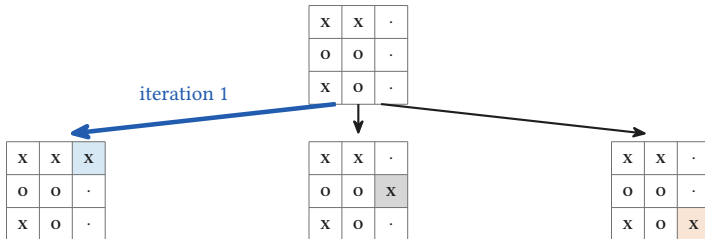
O answers in the middle row and wins

These three labels come from solving the game exactly. The search does not get them: it has to find out by playing.

Worked: the loop on the tree



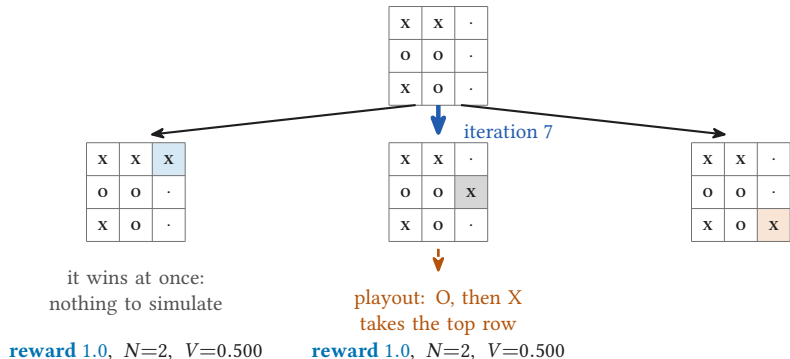
Worked: the loop on the tree



it wins at once:
nothing to simulate

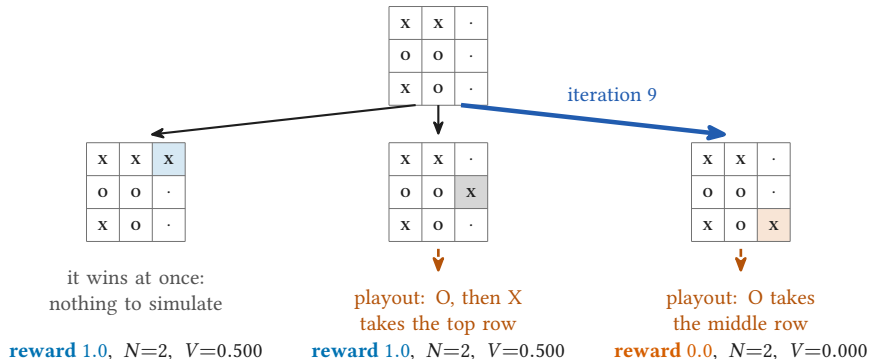
reward 1.0, $N=2$, $V=0.500$

Worked: the loop on the tree



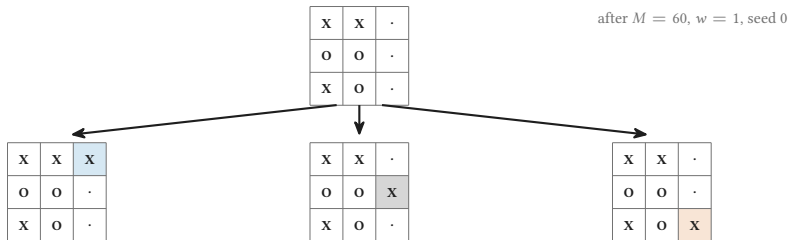
That middle move only draws. Random O blundered, so the playout won anyway: one playout is a sample, not the truth.

Worked: the loop on the tree



That middle move only draws. Random O blundered, so the playout won anyway: one playout is a sample, not the truth.

Worked: the loop on the tree



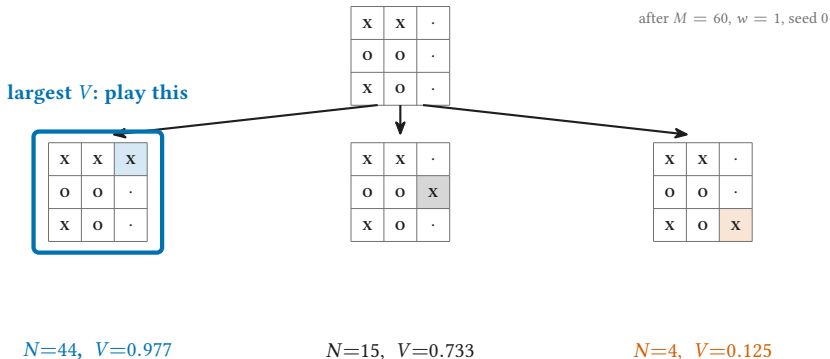
$N=44$, $V=0.977$

$N=15$, $V=0.733$

$N=4$, $V=0.125$

That middle move only draws. Random O blundered, so the playout won anyway: one playout is a sample, not the truth.

Worked: the loop on the tree



That middle move only draws. Random O blundered, so the playout won anyway: one playout is a sample, not the truth.

A real agent task: shopping on the web

WebShop. A shopping site with 1.18M real products. The agent is given a sentence and has to buy the right thing.

A real agent task: shopping on the web

WebShop. A shopping site with 1.18M real products. The agent is given a sentence and has to buy the right thing.

"i would like a 3 ounce bottle of bright citrus deodorant for sensitive skin, and price lower than 50.00 dollars"

A real agent task: shopping on the web

WebShop. A shopping site with 1.18M real products. The agent is given a sentence and has to buy the right thing.

"i would like a 3 ounce bottle of bright citrus deodorant for sensitive skin, and price lower than 50.00 dollars"

An action is a search or a click. The reward at the end is the share of the asked-for attributes the bought product actually has.

A real agent task: shopping on the web

WebShop. A shopping site with 1.18M real products. The agent is given a sentence and has to buy the right thing.

"i would like a 3 ounce bottle of bright citrus deodorant for sensitive skin, and price lower than 50.00 dollars"

An action is a search or a click. The reward at the end is the share of the asked-for attributes the bought product actually has.

method	score
one pass, no search	53.8
with search (LATS)	75.9
a human	82.1

A real agent task: shopping on the web

WebShop. A shopping site with 1.18M real products. The agent is given a sentence and has to buy the right thing.

"i would like a 3 ounce bottle of bright citrus deodorant for sensitive skin, and price lower than 50.00 dollars"

An action is a search or a click. The reward at the end is the share of the asked-for attributes the bought product actually has.

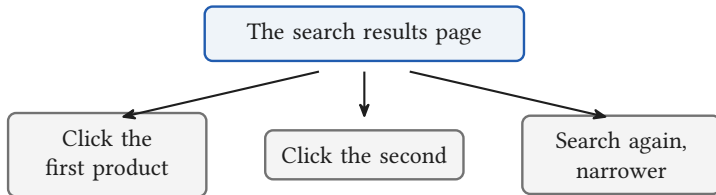
method	score
one pass, no search	53.8
with search (LATS)	75.9
a human	82.1

Zhou et al., LATS, ICML 2024, Table 6

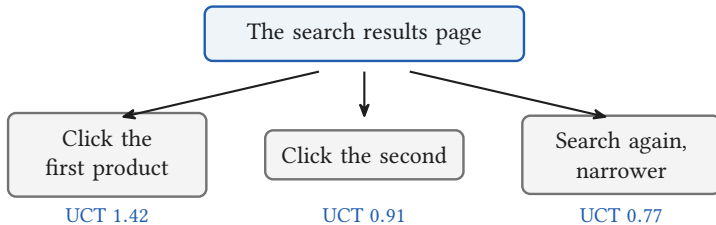
The same loop, on that task

The search results page

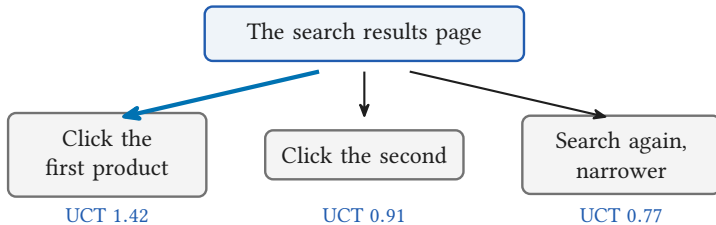
The same loop, on that task



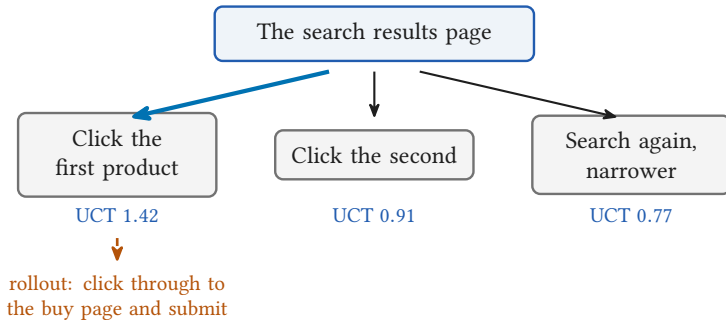
The same loop, on that task



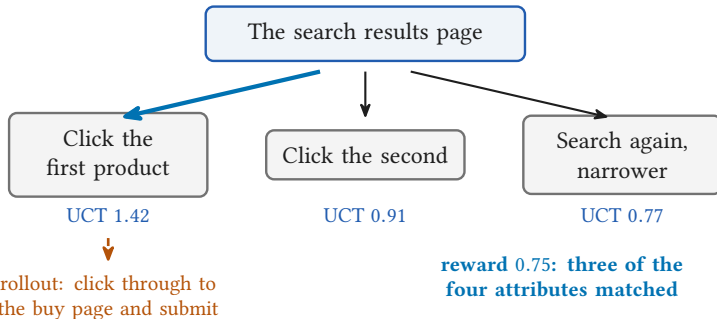
The same loop, on that task



The same loop, on that task

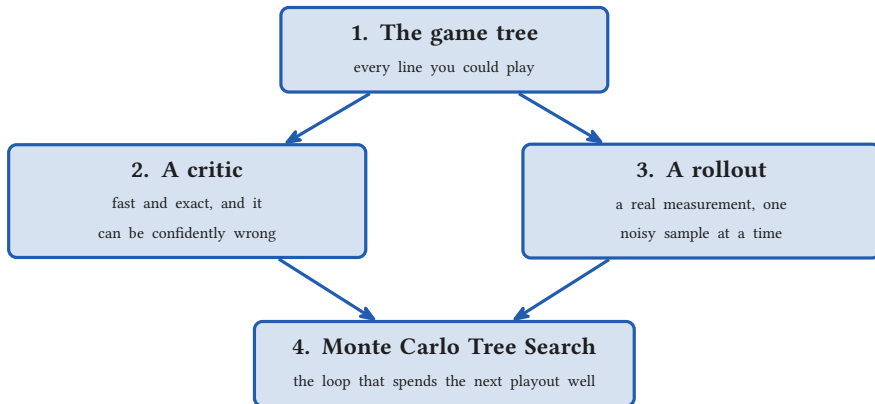


The same loop, on that task



These UCT values are illustrative, not a measured LATS run. At 10:00 a language model does the scoring.

Four things, and you have all four



At 10:00: the same search, with a language model in every box.